EXPERIMENT NO: 01

OBJECTIVE:

To evaluate the impact of AI medical robots (Nurabot, GR3, Moxy, Pepper) on reducing nurse workload and improving patient interaction through MATLAB simulation.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:

MATLAB software environment, AI Robots (Nurabot, Forier's GR3, Moxy, Pepper), Sensors (LiDAR, 3D cameras) , AI processors (NVIDIA Jetson), and Computer System.

THEORY:

AI medical robots are designed to assist healthcare professionals by automating routine and repetitive nursing tasks, thus reducing physical and cognitive workload. These robots can perform functions like guiding patients, delivering medication or supplies, monitoring vital signs, and facilitating communication between patients and caregivers. This relief of routine burden allows nurses to focus more on complex care activities and personalized patient interaction, thereby improving overall healthcare quality and patient satisfaction. MATLAB offers a robust software environment for simulating the operational behaviors of such AI medical robots. Using MATLAB, researchers can model the robots' sensory inputs, decision-making algorithms, and interaction protocols in a virtual healthcare setting. Essential components in this simulation include AI processors (such as NVIDIA Jetson), sensors like LiDAR and 3D cameras for environmental awareness, and the robots themselves programmed with AI for autonomous navigation and communication. The simulation enables assessment of various scenarios to quantify reductions in nurse workload by measuring task completion efficiency and the frequency of human intervention needed. Similarly, improvements in patient interaction are evaluated by modeling robot-patient communication dynamics and response times. MATLAB's visualization and data analysis tools facilitate detailed examination of these performance metrics, allowing researchers to fine-tune AI algorithms and robot behaviors before real-world deployment.

EXPERIMENT PROCEDURE:

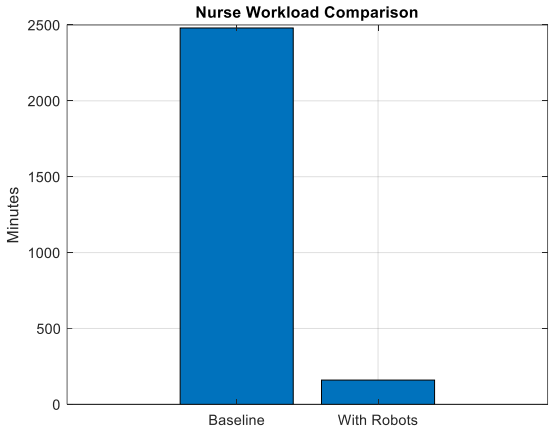
The experiment procedure begins by setting up the MATLAB simulation environment where AI medical robots such as Nurabot, GR3, Moxy, and Pepper are integrated with sensor models including LiDAR and 3D cameras, as well as AI processors like NVIDIA Jetson. The robots are programmed with navigation and interaction algorithms to emulate real-world nursing tasks in a virtual healthcare setting. Simulation runs are conducted to perform these tasks, during which nurse workload metrics and patient interaction parameters are continuously recorded. The collected data is then analyzed to quantify how the robots reduce workload and enhance patient engagement. Multiple simulation iterations are performed with varying parameters to optimize robot performance and ensure robustness before deployment in actual clinical environments. This continuous, iterative approach leverages MATLAB's powerful modeling and visualization capabilities to provide precise and actionable insights into the effectiveness of AI medical robots in healthcare. This type of simulation-based experiment helps validate the practical utility of AI robots and guides their further development and integration in medical settings.

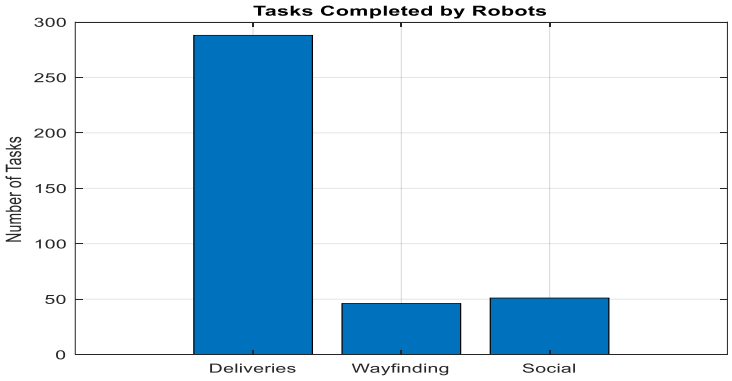
MATLAB CODE

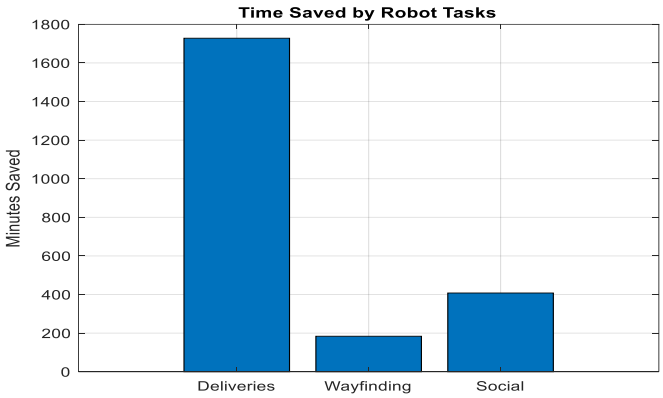
```
   AI ROBOT NURSE EXPERIMENT (Professional MATLAB Simulation)
   Evaluating Nurse Workload Reduction Using Medical Robots
   Author: Student Name
   Course: Biomedical / AI in Healthcare Lab
clc; clear; close all;

  SECTION 1 — PARAMETERS (Hospital Shift)
 Number of requests during a shift
numDeliveries      = 300;    % logistic tasks
numWayfinding      = 50;     % patient guidance
numSocialVisits    = 60;     % patient interaction task
 Nurse time cost per task (minutes)
Tn_delivery        = 6;
Tn_wayfinding      = 4;
Tn_social          = 8;

% Robot time cost per task (minutes)
Tr_delivery        = 2;
Tr_wayfinding      = 1.5;
Tr_social          = 4;

% Success rates of robots
P_delivery         = 0.96;
P_wayfinding       = 0.92;
P_social           = 0.85;

%% ------------------------------------------------------------
%  SECTION 2 — BASELINE (No Robots)
% ------------------------------------------------------------

nurseTime_baseline = numDeliveries*Tn_delivery + ...
                     numWayfinding*Tn_wayfinding + ...
                     numSocialVisits*Tn_social;

%% ------------------------------------------------------------
%  SECTION 3 — ROBOT SIMULATION
% ------------------------------------------------------------

% Successful tasks completed by robots
deliveries_done    = round(numDeliveries   * P_delivery);
wayfinding_done    = round(numWayfinding   * P_wayfinding);
social_done        = round(numSocialVisits * P_social);

% Time saved when robots replace nurse work
timeSaved_delivery = deliveries_done * Tn_delivery;
timeSaved_wayfind  = wayfinding_done * Tn_wayfinding;
timeSaved_social   = social_done    * Tn_social;

total_time_saved   = timeSaved_delivery + ...
                     timeSaved_wayfind  + ...
                     timeSaved_social;

% Robot working time (for hospital logistics calculation)
robot_work_time    = deliveries_done*Tr_delivery + ...
                     wayfinding_done*Tr_wayfinding + ...
                     social_done*Tr_social;

% Final workload after introducing robots
```

```
nurseTime_after    = nurseTime_baseline - total_time_saved;

% Percent reduction in nurse workload
percentReduction   = (total_time_saved / nurseTime_baseline) * 100;

%% ---------------------------------------------------------
%  SECTION 4 — DISPLAY RESULTS (Professional Output)
% ---------------------------------------------------------

fprintf("\n================================================\n");
fprintf("        AI ROBOT NURSE EXPERIMENT RESULTS\n");
fprintf("================================================\n");

fprintf("Baseline Nurse Time          : %.1f minutes\n",
nurseTime_baseline);
fprintf("Nurse Time With Robots       : %.1f minutes\n",
nurseTime_after);
fprintf("Total Time Saved             : %.1f minutes\n",
total_time_saved);
fprintf("Workload Reduction           : %.1f %%\n", percentReduction);
fprintf("Robot Working Time (Total)   : %.1f minutes\n",
robot_work_time);

fprintf("------------------------------------------------\n");
fprintf("Robot Performance:\n");
fprintf("  Deliveries Completed       : %d tasks\n", deliveries_done);
fprintf("  Wayfinding Completed       : %d tasks\n", wayfinding_done);
fprintf("  Social Visits Completed    : %d tasks\n", social_done);
fprintf("================================================\n\n");

%% ---------------------------------------------------------
%  SECTION 5 — GRAPHICAL RESULTS (Professional Plots)
% ---------------------------------------------------------

% 1. Nurse time comparison
figure;
bar([nurseTime_baseline, nurseTime_after], 'FaceColor','flat');
title('Nurse Workload Comparison');
ylabel('Minutes');
set(gca,'XTickLabel',{'Baseline','With Robots'});
grid on;

% 2. Robot task breakdown
figure;
bar([deliveries_done, wayfinding_done, social_done],
'FaceColor','flat');
title('Tasks Completed by Robots');
ylabel('Number of Tasks');
set(gca,'XTickLabel',{'Deliveries','Wayfinding','Social'});
grid on;

% 3. Time saved by category
figure;
bar([timeSaved_delivery, timeSaved_wayfind, timeSaved_social],
'FaceColor','flat');
title('Time Saved by Robot Tasks');
ylabel('Minutes Saved');
set(gca,'XTickLabel',{'Deliveries','Wayfinding','Social'});
grid on;
```

RESULTS







Command Window

```
==================================================
        AI ROBOT NURSE EXPERIMENT RESULTS
==================================================
Baseline Nurse Time          : 2480.0 minutes
Nurse Time With Robots       : 160.0 minutes
Total Time Saved             : 2320.0 minutes
Workload Reduction           : 93.5 %
Robot Working Time (Total)   : 849.0 minutes
--------------------------------------------------
Robot Performance:
  Deliveries Completed       : 288 tasks
  Wayfinding Completed       : 46 tasks
  Social Visits Completed    : 51 tasks
==================================================
```

PRE LAB QUESTIONS

1. What are the primary functions and capabilities of AI medical robots such as Nurabot, GR3, Moxy, and Pepper in healthcare settings?
2. How does MATLAB facilitate the simulation of AI medical robots and their interactions in a clinical environment?
3. What types of sensors and AI processors are commonly used to enable autonomous navigation and patient interaction in medical robots?
4. In what ways can AI medical robots contribute to reducing the workload of nursing staff?
5. What performance metrics and indicators are important to measure when evaluating the impact of AI robots on patient care quality?

POST LAB QUESTIONS

1. What effects did the AI medical robots have on reducing the nurse workload in the simulation?
2. How did the patient interaction improve when assisted by the AI medical robots during the simulation?
3. What limitations or challenges were observed in the MATLAB simulation of the AI medical robots?
4. How can the simulation results guide real-world implementation of these AI medical robots in healthcare settings?
5. What further improvements or additional features could be incorporated into the AI robots or simulation for enhanced performance?

EXERCISE

a. Write a report to summarizing the objectives and methodology of the experiment where AI medical robots (Nurabot, GR3, Moxy, Pepper) were simulated in MATLAB to evaluate their impact on nurse workload and patient interaction.

b. Describe your observations on how the robots influenced nursing efficiency and patient engagement based on simulation data.

EXPERIMENT NO: 02

OBJECTIVE:
To observe and analyze the operational mechanisms of key robotic applications, including robotic-assisted surgery, rehabilitation robotics, telepresence, pharmacy automation, and robotic prosthetics.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Computer with CAD Software (e.g., SolidWorks, Fusion 360), Prototyping Kits (e.g., LEGO Mindstorms, VEX Robotics, Arduino/Raspberry Pi with sensors and actuators), Standard Workshop Tools and Materials (3D printer, foam board, motors, etc.), Simulation Software (where physical apparatus is unavailable, e.g., for surgical or therapy robot simulation)

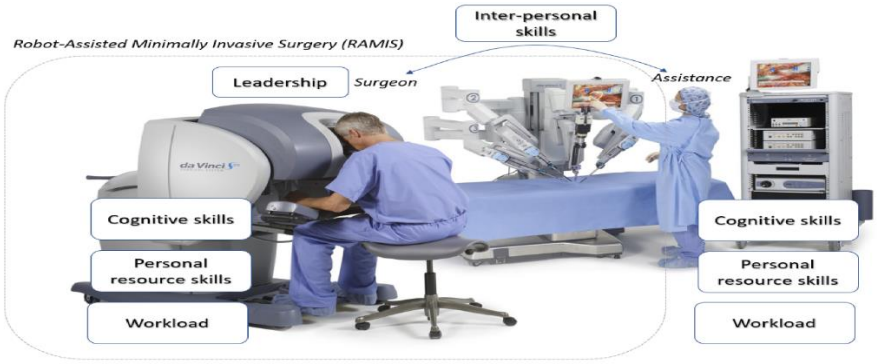
THEORY:
The integration of advanced robotics into healthcare represents a paradigm shift in medical practice. These technologies leverage precision engineering, sensors, and intelligent control systems to augment the capabilities of healthcare professionals. The core theory underpinning this field is that robotics can enhance procedural precision, enable minimally invasive techniques, provide consistent and quantifiable therapeutic interventions, automate repetitive tasks, and restore lost physiological functions.

Key application areas include:
Robotic-Assisted Surgery: Systems like the da Vinci Surgical System provide surgeons with enhanced dexterity, 3D high-definition vision, and tremor filtration, leading to smaller incisions, reduced blood loss, and faster patient recovery.

Rehabilitation Robotics: Robotic exoskeletons and assistive devices deliver high-intensity, repetitive, and data-driven therapy, aiding patients in regaining motor skills and mobility after neurological events like strokes or spinal cord injuries.
Telepresence Robots: These mobile, remote-controlled platforms facilitate virtual consultations and patient monitoring, breaking down geographical barriers and improving access to specialist care in rural or underserved areas.

Pharmacy Automation: Robotic dispensing systems and inventory management robots minimize human error in medication preparation, enhancing patient safety and streamlining pharmacy workflow.
Robotic Prosthetics: Advanced prosthetic limbs utilize myoelectric or neural interfaces to interpret the user's intent, allowing for more natural and intuitive control over movement.

EXPERIMENT PROCEDURE:
Initially, students engage in an extensive literature review covering scholarly articles, case studies, and technical specifications to understand the engineering principles and patient care impacts associated with major robotic applications such as robotic-assisted surgery, rehabilitation devices, telepresence systems, pharmacy automation, and advanced prosthetics. Following this, the design and operational characteristics of these robotic systems are explored using CAD software to develop 3D models, allowing students to visualize and modify components cogent to healthcare challenges. Subsequently, students undertake the hands-on task of assembling robotic prototypes using kits like LEGO Mindstorms or Arduino-based platforms, integrating sensors and actuators to perform specific functionalities mimicking real robotic systems. These prototypes are then programmed and tested, focusing on controlled movements, repeatability, and system responsiveness to simulate clinical tasks effectively. Parallel to physical prototyping, simulation software may be employed to recreate surgical or rehabilitative scenarios, providing insights into robotic system behavior during complex procedures. Throughout the experiment, quantitative data related to performance metrics such as precision, speed, and error rates are collected and analyzed to evaluate the benefits of robotic assistance compared to traditional methods. Finally, students assess the overall impact of robotic solutions on patient outcomes, healthcare efficiency, and accessibility, while recognizing limitations and future development pathways. The comprehensive findings and design documentation are compiled into a detailed report and presented, fostering critical appraisal and deeper understanding of robotics' transformative role in healthcare



Sample Results Table for Prototype Testing:

Test Parameter	Expected Outcome	Measured Outcome	Analysis / Deviation
Range of Motion (Degrees)	0 - 90°	0 - 85°	Slight friction at joint limit caused 5° loss.
Task Completion Time (s)	< 10 s	12 s	Motor torque was insufficient for smooth movement.
Lifting Capacity (g)	200 g	150 g	Grip mechanism slipped at higher weights.

Test Parameter	Expected Outcome	Measured Outcome	Analysis / Deviation
User Feedback Score (1-5)	4	3.5	Users found controls unintuitive.

PRE LAB QUESTIONS

1. What are the primary types of robots used in healthcare, and how do they improve medical procedures and patient care?
2. How does robotic-assisted surgery enhance precision and reduce recovery times compared to traditional surgery?
3. In what ways do rehabilitation robots aid patients in regaining mobility and functional abilities after injury or illness?
4. What role do telepresence robots play in increasing access to healthcare services in remote or underserved areas?
5. How does automation in pharmacy systems contribute to patient safety and improve the efficiency of medication management?

POST LAB QUESTIONS

1. How did the robotic prototype or simulation improve the precision and efficiency of the healthcare task it was designed for?
2. What were the key challenges faced while designing and programming the robotic system, and how were they addressed?
3. In what ways can robotic automation reduce human errors and enhance patient safety in healthcare settings?
4. How does the integration of sensors and feedback mechanisms impact the performance and adaptability of healthcare robots?
5. Based on the experiment, what are the potential limitations or ethical considerations when implementing robotic technology in patient care?

EXERCISE

a. Explain the process you followed to design, prototype, and test your robotic solution.
b. Discuss the observed impact of robotic assistance on patient care metrics such as precision, safety, and operational efficiency during the experiment.

EXPERIMENT NO: 03

OBJECTIVE:
Continuous Stress Detection in Nurses Using Wearable Sensors with Machine Learning Approach

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Empatica E4 wristband or similar wearable device to record physiological signals (electro dermal activity, heart rate, skin temperature), Smartphone app for data transmission and survey feedback, Laptop or cloud environment (Google Colab) with Python 3.7+ and libraries: pandas, numpy, scikit-learn, matplotlib, seaborn, tensorflow/keras, Dataset (merged_data.csv) containing synchronized sensor data and stress labels.

THEORY:
Stress triggers physiological changes controlled by the autonomic nervous system, which can be tracked via wearable sensors measuring biosignals such as Electrodermal Activity (EDA), Heart Rate (HR), and skin temperature. EDA indicates changes in sweat gland activity linked to stress, HR reflects cardiovascular response to stress, and temperature can indicate thermoregulatory changes during stress. Machine learning algorithms can classify and predict stress levels by analyzing patterns in these physiological signals integrated with contextual data.

EXPERIMENT PROCEDURE:

1. Data Preparation:
    a. Load the dataset with physiological signals and timestamps.
    b. Engineer features using rolling window statistics (mean, std, min, max) and temporal derivatives (delta).
    c. Handle missing data and extract input features (e.g., EDA_roll_mean, HR_roll_std).
2. Data Processing:
    a. Split data into training and testing sets, preserving class distribution.
    b. Normalize features using Standard Scaler to enhance model convergence.
    c. Reshape data for CNN input dimension requirements.
3. Model Building & Training:
    a. Define a 1D convolutional neural network with Conv1D, pooling, batch normalization, dropout, and dense layers.
    b. Compile the model with Adam optimizer and sparse categorical cross-entropy loss.
    c. Train the model over multiple epochs with a validation split.
4. Model Evaluation:
    a. Evaluate the trained model on the test set using accuracy, classification report, and confusion matrix.
    b. Analyze model performance across stress categories.
5. Saving Results:
    a. Save the trained model and scaler objects for future deployment and inference.

CODE

```
!git clone https://github.com/niaz1971/Stress-Detection-in-Nurses-Using-Wearable-Sensors.git
cd Stress-Detection-in-Nurses-Using-Wearable-Sensors/
!pip install lime kaggle
# Step 2: Set up Kaggle API for dataset download
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
# Step 3: Download and unzip malaria cell images dataset
!kaggle datasets download -d priyankraval/nurse-stress-prediction-wearable-sensors
!unzip -q nurse-stress-prediction-wearable-sensors.zip
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam

# Load CSV safely
data = pd.read_csv('merged_data.csv', low_memory=False)
data.columns = data.columns.str.strip()  # remove spaces in headers

# Confirm columns
print("Columns in CSV:", data.columns.tolist())

# Use actual sensor columns and label
sensor_columns = ['EDA', 'HR', 'TEMP']  # physiological sensors relevant to stress
label_column = 'label'                  # binary stress label column

# Check for missing columns
missing_cols = [col for col in sensor_columns + [label_column] if col not in data.columns]
if missing_cols:
    raise KeyError(f"Columns missing from CSV: {missing_cols}")

# Convert sensor columns to numeric, coerce errors to NaN
for col in sensor_columns:
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Drop rows with NaN values in sensor columns or label column
data.dropna(subset=sensor_columns + [label_column], inplace=True)

# Extract features and labels
X = data[sensor_columns].values
y = data[label_column].values

# Define time_steps window size
time_steps = 100  # adjust based on your dataset's sampling/window
num_channels = len(sensor_columns)
```

```
# Calculate total full samples
total_samples = X.shape[0] // time_steps

# Trim excess rows to fit full samples
X = X[:total_samples * time_steps]
y = y[:total_samples * time_steps]

# Reshape features to (samples, time_steps, channels)
X = X.reshape(total_samples, time_steps, num_channels)

# Reshape labels to (samples, time_steps) and pick last label for each sample
y = y.reshape(total_samples, time_steps)
y = y[:, -1]

# Normalize features
scaler = StandardScaler()
X_reshaped = X.reshape(-1, num_channels)
X_scaled = scaler.fit_transform(X_reshaped)
X = X_scaled.reshape(total_samples, time_steps, num_channels)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Build 1D CNN model
def build_cnn(input_shape):
    model = Sequential([
        Conv1D(32, kernel_size=3, activation='relu', input_shape=input_shape),
        BatchNormalization(),
        MaxPooling1D(2),
        Conv1D(64, kernel_size=3, activation='relu'),
        BatchNormalization(),
        MaxPooling1D(2),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=Adam(0.001),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model

model = build_cnn((time_steps, num_channels))
model.summary()

# Train for 3 epochs
model.fit(X_train, y_train, epochs=3, batch_size=32, validation_split=0.2)

# Predict stress
predictions = model.predict(X_test)
```

```
predicted_labels = (predictions > 0.5).astype(int)
print("Predicted stress labels on test data:", predicted_labels.flatten())

# After predicting labels as 0 or 1 (0 = No Stress, 1 = Mild Stress)
predicted_labels = (predictions > 0.5).astype(int).flatten()

for i, label in enumerate(predicted_labels):
    if label == 0:
        print(f"Sample {i + 1}: Nurse is in NO Stress condition.")
    else:
        print(f"Sample {i + 1}: Nurse is in Mild Stress condition.")
```

RESULTS

```
Sample 22946: Nurse is in Mild Stress condition.
Sample 22947: Nurse is in Mild Stress condition.
Sample 22948: Nurse is in Mild Stress condition.
Sample 22949: Nurse is in Mild Stress condition.
Sample 22950: Nurse is in Mild Stress condition.
Sample 22951: Nurse is in Mild Stress condition.
Sample 22952: Nurse is in Mild Stress condition.
Sample 22953: Nurse is in Mild Stress condition.
Sample 22954: Nurse is in Mild Stress condition.
Sample 22955: Nurse is in Mild Stress condition.
Sample 22956: Nurse is in Mild Stress condition.
Sample 22957: Nurse is in Mild Stress condition.
```

PRE LAB QUESTIONS

1. What physiological signals are typically monitored by wearable sensors to detect stress in individuals?
2. Why is electrodermal activity (EDA) considered a significant biomarker for stress detection compared to heart rate or skin temperature?
3. How do machine learning models like convolutional neural networks (CNNs) help in predicting stress from sensor data?
4. What are the advantages of using rolling window statistics (mean, standard deviation) as features in time-series physiological data?
5. How does the validation survey from nurses complement the physiological data in improving the accuracy of stress detection?

POST LAB QUESTIONS

1. Which physiological signals were most effective in predicting nurse stress levels and why?
2. How did the convolutional neural network perform in classifying stress, and what metrics indicate its accuracy?
3. What role did feature engineering (rolling window statistics) play in improving model performance?
4. What challenges or limitations did you encounter while using wearable sensor data for stress prediction?
5. How can continuous stress monitoring through wearable sensors improve nurse well-being and healthcare outcomes?

EXERCISE

Write one-page report describing your observations during the lab session, using your own words.

EXPERIMENT NO: 04

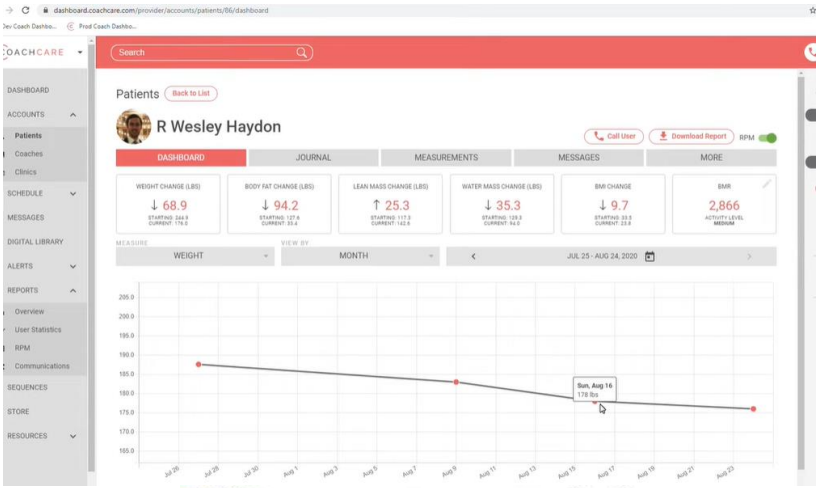
OBJECTIVE:
To perform the Remote Patient Monitoring (RPM) System Demonstration and Operation.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Wireless biometric monitoring devices (e.g., body composition scale, blood pressure monitor, glucometer), CoachCare RPM platform (software and app), Personal computer or provider workstation, Patient smartphone or tablet with CoachCare app installed, and Internet connection for data transmission.

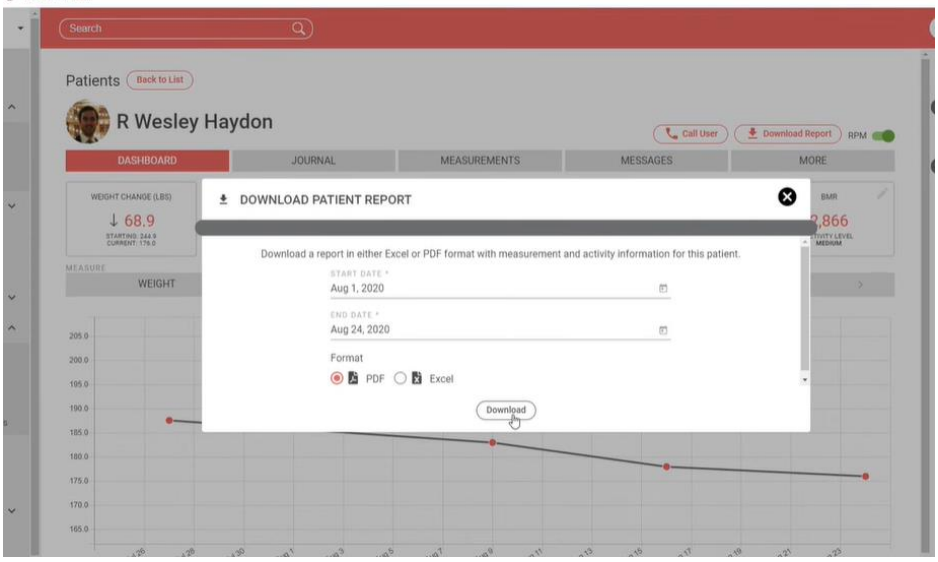
THEORY:
Remote Patient Monitoring (RPM) is a telehealth service that enables healthcare providers to monitor patients' vital health data from a distance using connected medical devices. RPM supports value-based care by facilitating proactive patient health management, reducing hospital visits, and lowering healthcare costs. The system collects biometric data such as blood pressure, glucose levels, and body composition from patients in real-time. This data is transmitted via wireless devices to a central platform accessible by healthcare staff for continuous monitoring and timely intervention.

EXPERIMENT PROCEDURE:
1. Obtain patient consent for RPM services and enroll the patient in the system.
2. Order required RPM services through appropriate administrative channels.
3. Provide the patient with designated wireless monitoring devices tailored to their health needs.
4. Instruct the patient to download and install the CoachCare app on their smartphone or tablet.
5. Guide the patient to connect their wireless devices to the app for automatic data capture.
6. Activate the patient account on the CoachCare RPM platform from the provider's side.
7. Monitor incoming biometric data via the platform dashboard.
8. Document provider time spent monitoring the patient, generating clinical reports as needed for healthcare records.
9. Use billing and eligibility summary reports for managing claims processing.

RESULTS
1. Successful transmission of biometric data from patient devices to the RPM platform.
2. Continuous, real-time remote health monitoring enabled without in-person visits.
3. Automatic documentation of provider monitoring activities for clinical and billing purposes.
4. Enhanced patient management through timely data review and intervention by healthcare staff.
5. Streamlined claims processing supported by comprehensive reporting tools.


Search
Patients
Back to List
R Wesley Haydon
Call User
Download Report
RPM
DASHBOARD
JOURNAL
MEASUREMENTS
MESSAGES
MORE
WEIGHT CHANGE (LBS)
↓ 68.9
BODY FAT CHANGE (LBS)
↓ 94.2
LEAN MASS CHANGE (LBS)
↑ 25.3
WATER MASS CHANGE (LBS)
↓ 35.3
BMI CHANGE
↓ 9.7
BMR
2,866
WEIGHT
MONTH
JUL 25 - AUG 24, 2020
Sun, Aug 16
178 lbs


Wes' iPhone
4:57
Progress
Progress
August 2020
MON 24
TUE 25
WED 26
THU 27
FRI 28
SAT 29
SUN 30
All
Selected
BODY COMPOSITION
Weight
176 pounds
Body Fat
19 %
BMI
23.8
VITALS
Blood Pressure
135 / 80 mmHg
Body Temperature
99 fahrenheit
Respiration Rate
15 /min
Home
Progress
Contact
Schedule


Search
Patients
Back to List
R Wesley Haydon
Call User
Download Report
RPM
DASHBOARD
JOURNAL
MEASUREMENTS
MESSAGES
MORE
DOWNLOAD PATIENT REPORT
Download a report in either Excel or PDF format with measurement and activity information for this patient.
START DATE *
Aug 1, 2020
END DATE *
Aug 24, 2020
Format
PDF
Excel
Download

CoachCare Patient Clinical Report

Dates of Service: August 1, 2020 - August 24, 2020

Patient Information

First Name	Last Name	Date of Birth	Organization ID	Organization Name
R Wesley	Haydon	02/18/1987	4066	CoachCare Clinic

Patient Care Plan

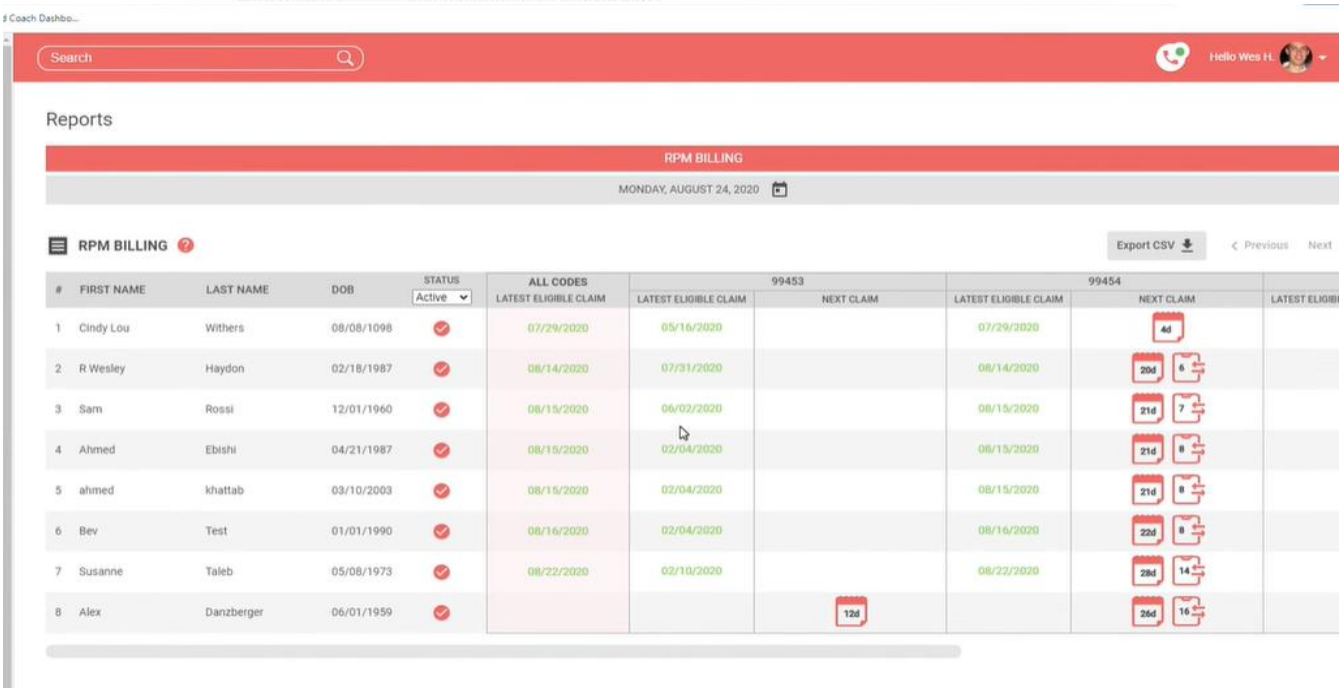
Requirements	Status
Care Plan Type	RPM
Status	Active
Activation Date	07/15/2020
Target Deactivation Date	
Actual Deactivation Date	
Reason for Deactivation	
Consent Obtained	07/15/2020
Face-to-Face within 12 months	Yes
Device Supplied	07/15/2020
Device Education Provided	07/15/2020

Patient Data

Metric	Unit	Start	End	Unit Change	% Change	Total Transmission Days
Weight	lb	183	176	-7	-4%	24
BMI		24.8	23.8	-0.9	-4%	24
Body Fat	%	20	19	-1	-5%	24
BP Sys	mmHg	140	135	-5	-4%	6
BP Dia	mmHg	85	80	-5	-6%	6
Temperature	F	99	99	32	0%	1
Exercise	min / week	40	40	0	0%	1

Provider Activity

Activity Type	Patient Interactions	Time (min)



PRE LAB QUESTIONS

1. What is the purpose of remote patient monitoring and how does it improve patient care?
2. What types of biometric data are commonly collected using remote patient monitoring devices?
3. How does the remote patient monitoring system ensure secure transmission and privacy of patient data?
4. What steps are involved in setting up a patient for remote monitoring using wireless devices and a mobile app?
5. How can healthcare providers use the data collected from remote monitoring to make clinical decisions and improve outcomes?

POST LAB QUESTIONS

1. What are the key benefits of using remote patient monitoring in managing chronic diseases?
2. How did the system ensure continuous and accurate data transmission from patient devices during the experiment?
3. What challenges or limitations did you observe in the use of wireless devices for remote monitoring?
4. How can healthcare providers use the data collected through RPM to prevent hospital readmissions?
5. What measures are necessary to protect patient privacy and data security in remote patient monitoring systems?

EXERCISE

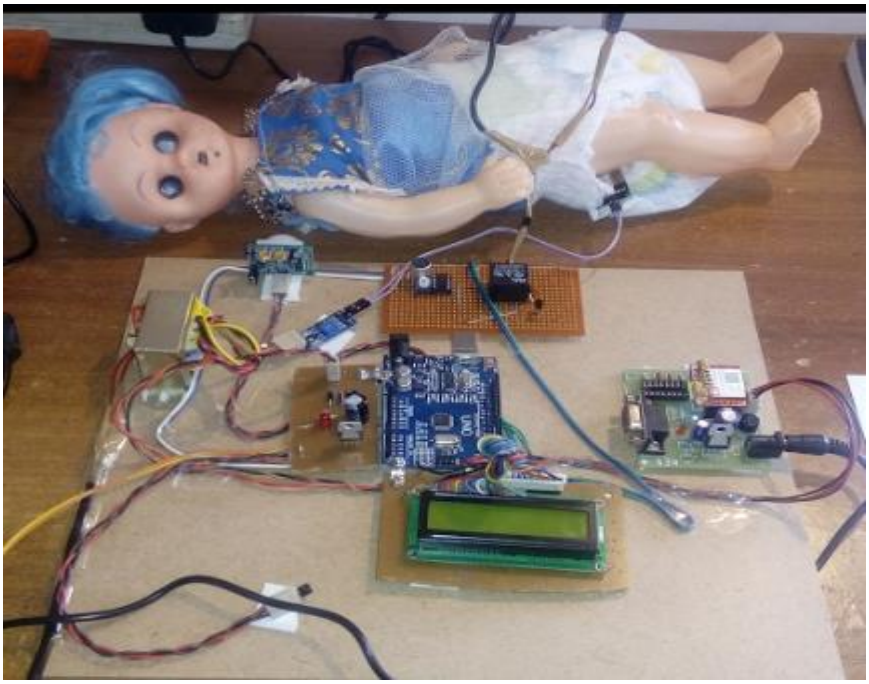
1. Describe the workflow of remote patient monitoring from data acquisition to clinical decision-making.
2. What are the key vital signs measurable via the monitoring devices used in the experiment?
3. Discuss how patient privacy and data security are maintained during remote monitoring.

EXPERIMENT NO: 05

OBJECTIVE:
To observe the Implementation of an IoT-Based Child Monitoring System with Real-Time Video Streaming.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Microcontroller ESP32-CAM Module, Integrated Wi-Fi, camera, ESP32-CAM board, power Supply, Computer with Arduino IDE, and smartphone.

THEORY:
The design and implementation of an IoT-based child monitoring system with real-time video streaming is an innovative approach to enhancing child safety and security. Utilizing the Internet of Things (IoT), this system integrates various sensors and devices such as cameras, GPS modules, temperature sensors, and microcontrollers to continuously monitor a child's environment and vital signs. The system collects real-time data, including location, body temperature, and activity, which is transmitted through wireless networks to a cloud-based platform accessible to parents or guardians via a smartphone application or web interface. One of the critical features of this system is the real-time video streaming capability, allowing caregivers to visually supervise children remotely, ensuring their safety even when physically apart. The IoT infrastructure supports creating safe zones with geofencing technology that alerts caregivers instantly if the child moves beyond predefined boundaries, thereby enhancing preventive care. This child monitoring solution emphasizes remote surveillance, continuous monitoring, and quick emergency notification, which facilitates timely intervention in critical situations such as falls, abnormal vital signs, or unauthorized movement. Additionally, data analytics integrated into the system can identify behavioral trends and health patterns, enabling proactive safety measures. Privacy and data security are prioritized through encrypted data transmission protocols and secure cloud storage. Overall, this IoT-based child monitoring system embodies the principles of digital transformation by leveraging connected technologies for healthcare and safety applications. It empowers parents and caregivers with real-time information and peace of mind, representing a significant advancement in child care and protection technology.



This system is a classic example of a client-server IoT architecture. The ESP32-CAM acts as the IoT Device/Server. It is a physical "thing" that senses its environment (via the camera) and makes that data available on the internet. The Android smartphone acts as the Client, which requests and consumes the data (the video stream) from the server. The ESP32-CAM is a low-cost, compact module that combines an ESP32-S microcontroller with an OV2640 camera. Its key features include a built-in Wi-Fi module, sufficient processing power to handle image capture and compression, and GPIO pins to connect peripherals, making it ideal for video streaming projects. The system uses MJPEG (Motion JPEG) for video streaming. Unlike more complex video codecs, MJPEG works by capturing a sequence of JPEG image frames from the camera, compressing them individually, and sending them sequentially over the network. The client (Android app) receives this stream of JPEGs and displays them rapidly one after another, creating the illusion of real-time motion video. This method is efficient for microcontrollers as it avoids the high computational cost of inter-frame compression.

WORKING PRINCIPLE:

1. Initialization: Upon powering up, the ESP32-CAM runs the uploaded firmware. It first connects to the pre-configured Wi-Fi network.

2. Server Startup: Once connected, it starts a web server and obtains a local IP address. This server has a specific endpoint (e.g., /video) that hosts the MJPEG stream.

3. Streaming: The camera continuously captures frames, which are encoded into JPEGs and sent over the network via the HTTP protocol.

4. Client Connection: The user opens the Android app and enters the ESP32's IP address. The app sends an HTTP request to the video stream endpoint.

5. Remote Viewing: The app receives the stream of JPEG images and decodes and renders them on the screen, providing a live video feed to the user.

6. Circuit Diagram and Setup

6.1. Hardware Connections:

The ESP32-CAM module requires a precise connection for programming and power. Using a USB-to-UART converter (like an FTDI programmer):

ESP32-CAM Pin	USB-to-UART Converter Pin
GND	GND
5V	5V
U0R	TX
U0T	RX
GPIO 0	GND (During programming only)

Important Note: The GPIO 0 pin must be connected to GND while uploading the code to put the board into programming mode. After the code is uploaded, remove this connection and reset the board for it to run normally.

6.2. Setup:

1. Connect the ESP32-CAM to the USB-to-UART converter as per the table above, with GPIO 0 connected to GND.
2. Connect the converter to the computer via USB.

EXPERIMENT PROCEDURE:

Step 1: Software Installation and Configuration

1. Install the Arduino IDE on your computer.
2. Add the ESP32 board support via the Boards Manager using this URL: https://dl.espressif.com/dl/package_esp32_index.json
3. Install the necessary libraries (e.g., ESP32 library, Webserver library, OV2640 camera driver).

Step 2: Firmware Development and Upload

1. In the Arduino IDE, select the "AI Thinker ESP32-CAM" board from the board menu.
2. Write/Copy the code that:
   - Includes the necessary Wi-Fi and camera libraries.
   - Sets the Wi-Fi credentials (SSID and Password).
   - Initializes the camera with a specific frame size (e.g., VGA: 640x480).
   - Starts a web server that sets up an MJPEG stream at a URL like http://[ESP_IP]/video.
3. With GPIO 0 connected to GND, click the upload button to flash the code to the ESP32-CAM.

Step 3: Network Configuration

1. After a successful upload, remove the GPIO 0 to GND connection.
2. Open the Serial Monitor at a baud rate of 115200.
3. Reset the ESP32-CAM board. The Serial Monitor will display the board's progress, including the assigned IP Address once it connects to Wi-Fi. Note down this IP address.

Step 4: Android Application Setup

1. Install the provided "IoT Camera" APK file on the Android smartphone or develop a basic app with a WebView/VideoView component.
2. Open the application.
3. In the app's settings or connection screen, enter the IP address of the ESP32-CAM noted in the previous step (e.g., http://192.168.1.15/video).

Step 5: Testing and Observation

1. Point the ESP32-CAM camera towards an area you wish to monitor.
2. On the smartphone, initiate the connection from within the app.
3. Observe the live video feed appearing on the smartphone screen.

RESULTS:

The system connectivity was successfully established as the ESP32-CAM connected to Wi-Fi and printed its IP address in the Arduino IDE's Serial Monitor. This confirmed that the device was properly connected to the network and was ready for streaming. The video streaming functionality was verified by accessing the camera's IP address URL in a web browser, where a clear live video feed from the camera was observed, indicating proper initialization and real-time streaming capability. The Android app also successfully displayed the live video feed, providing a smooth and clear real-time video stream with an approximate frame rate of 5-10 FPS, confirming the app's functionality in displaying the data received from the ESP32-CAM. Additionally, remote monitoring was tested by moving the smartphone to different locations within the Wi-Fi range, and the video feed remained stable and continuous, demonstrating reliable wireless communication and effective data transmission for remote caregiving. Overall, the observations and results confirmed that the IoT-based child monitoring system with real-time video streaming met the intended functional objectives, ensuring connectivity, video quality, app integration, and remote accessibility.

PRE LAB QUESTIONS

1. What is the primary objective of the IoT-based child monitoring system with real-time video streaming?
2. Which hardware components are essential for implementing the ESP32-CAM based monitoring system?
3. How does the MJPEG video streaming method work in the context of the ESP32-CAM?
4. What is the role of the ESP32-CAM board and the Android smartphone in the client-server architecture of this system?
5. What are the key steps involved in setting up the ESP32-CAM module and uploading the firmware using the Arduino IDE?

POST LAB QUESTIONS

1. Explain how the ESP32-CAM module establishes a connection to the Wi-Fi network and starts the video streaming server.
2. What challenges might arise in maintaining a stable real-time video stream over Wi-Fi, and how can these be mitigated?
3. Describe why the GPIO 0 pin must be connected to GND during firmware upload and disconnected afterwards.
4. Discuss the advantages and limitations of using MJPEG streaming for this IoT video monitoring system compared to other video compression methods.
5. How can security and privacy be ensured when transmitting video data from the child monitoring system over the internet?

EXERCISE

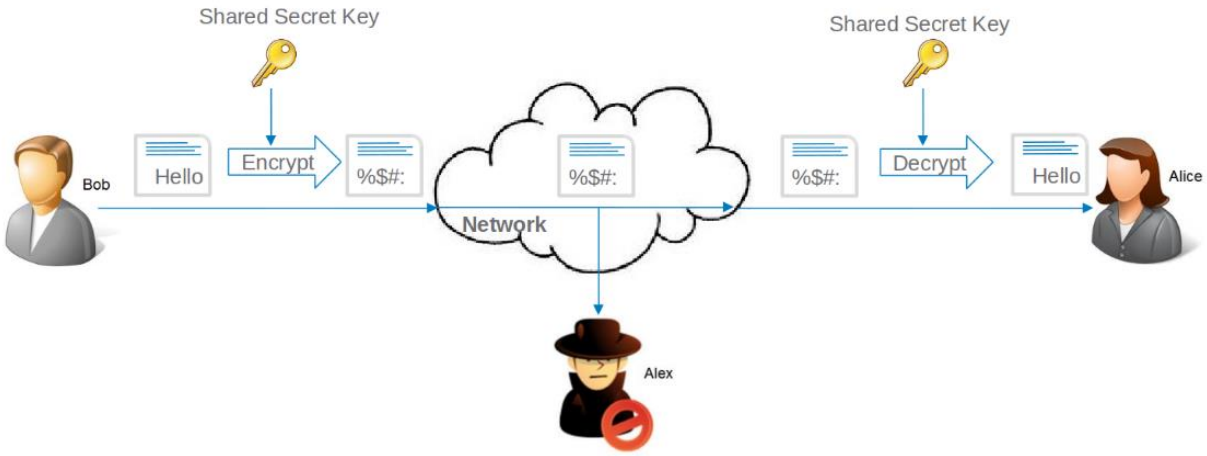
Write a report on the design and implementation of an IoT-based child monitoring system with real-time video streaming. Include the system architecture, hardware and software components used, and the working principle. Discuss the significance of IoT in enhancing child safety and healthcare, as well as the challenges related to data privacy, security, and real-time monitoring. During the lab experiment, observe and describe the ESP32-CAM's Wi-Fi connectivity, the process of obtaining the IP address, video stream initialization, video feed quality on the app, and remote monitoring stability across different Wi-Fi locations.

EXPERIMENT NO: 06

OBJECTIVE:
To implement a simple encryption and decryption algorithm for data security purpose using Python.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Computer with Python installed (preferably Python 3.x), Python cryptography library (pip install cryptography), Text editor or Integrated Development Environment (IDE) such as VSCode, PyCharm, or Jupyter Notebook.

THEORY:
Symmetric key cryptography uses a single shared secret key for both encryption and decryption. The sender encrypts the plaintext using the key to produce ciphertext. The receiver, possessing the same secret key, decrypts the ciphertext to obtain the original plaintext. This method is efficient for securing data but requires secure key distribution. The "Fernet" module in Python's cryptography library implements authenticated encryption using AES in CBC mode with a random IV and HMAC for integrity, providing both confidentiality and authentication.



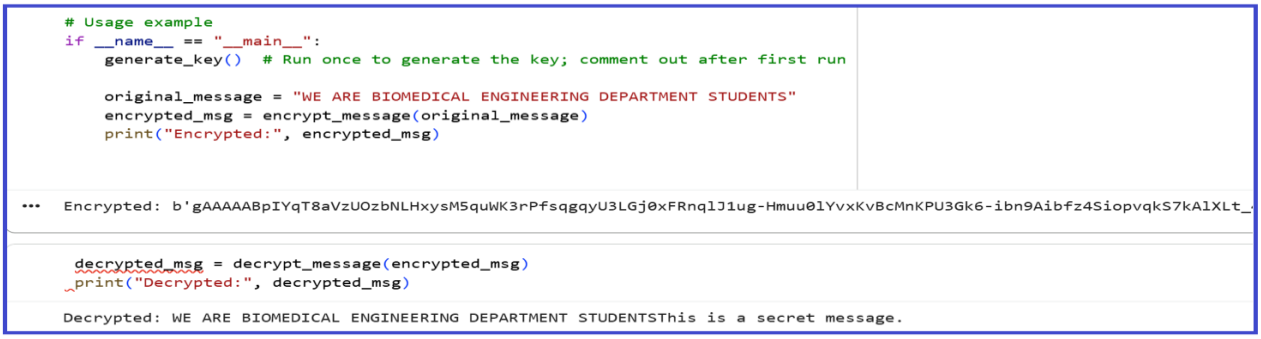
EXPERIMENT PROCEDURE:

- Install the cryptography library if not already installed (!pip install cryptography)
- Generate a secret key to be used for encryption and decryption.
- Write a Python script to:
- Load the encryption key.
- Encrypt a plaintext message.
- Decrypt the encrypted message back to the original.
- Run the script to encrypt a sample message.
- Print the encrypted message (ciphertext).
- Print the decrypted message to verify correctness.
- Sample Python Code for Encryption/Decryption

```
from cryptography.fernet import Fernet

def generate_key():
    key = Fernet.generate_key()
    with open("secret.key", "wb") as key_file:
        key_file.write(key)

def load_key():
    return open("secret.key", "rb").read()

def encrypt_message(message: str) -> bytes:
    key = load_key()
    f = Fernet(key)
    return f.encrypt(message.encode())

def decrypt_message(encrypted_message: bytes) -> str:
```

```
    key = load_key()
    f = Fernet(key)
    return f.decrypt(encrypted_message).decode()

if __name__ == "__main__":
    generate_key()  # Run once to create the key file

    original_message = "This is a secret message."
    encrypted_msg = encrypt_message(original_message)
    print("Encrypted:", encrypted_msg)

    decrypted_msg = decrypt_message(encrypted_msg)
print("Decrypted:", decrypted_msg)
```

RESULTS



PRE LAB QUESTIONS

1. What is symmetric key cryptography, and how does it differ from asymmetric key cryptography?
2. Why is key management important in symmetric encryption?
3. What role does the secret key play in encryption and decryption processes?
4. Describe the importance of using initialization vectors (IV) in symmetric encryption algorithms.
5. What are the potential risks or vulnerabilities associated with symmetric key cryptography?

POST LAB QUESTIONS

1. Explain how the encryption and decryption process works in the experiment.
2. Why is it important to keep the encryption key secure, and what could happen if it is compromised?
3. How does the use of the cryptography library's Fernet module ensure both confidentiality and data integrity?
4. Describe any challenges you faced while implementing the encryption and decryption code.
5. How can symmetric key cryptography be applied in real-world scenarios? Discuss its advantages and limitations.

EXERCISE

Write a Python program to securely encrypt and decrypt your name and roll number.

EXPERIMENT NO: 07

OBJECTIVE:

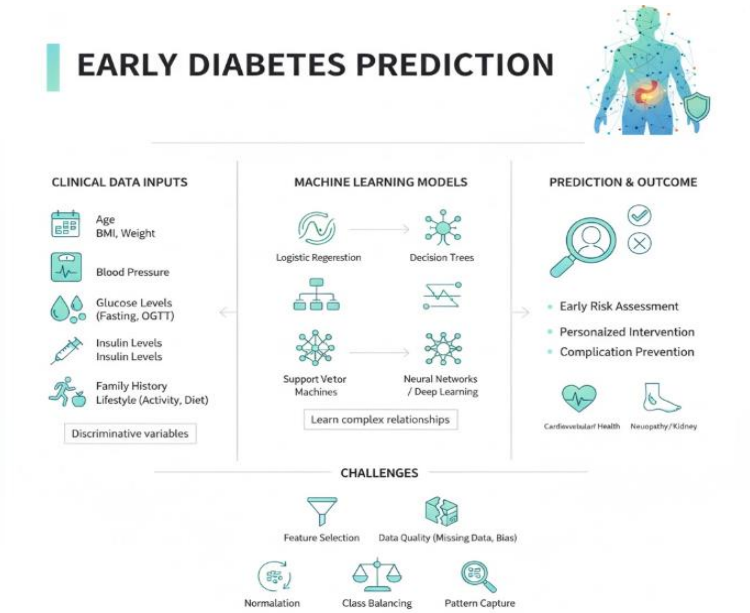
Diabetes Prediction Using Deep Learning with TensorFlow and Keras

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:

Google Colab, Internet connectivity, Operating Systems, Windows, Python (latest stable version), TensorFlow and Keras.

THEORY:

Diabetes mellitus is a chronic metabolic disorder characterized by high blood glucose levels resulting from defects in insulin secretion, insulin action, or both. Early prediction of diabetes is critical for effective management and prevention of its complications such as cardiovascular diseases, neuropathy, and kidney failure. Predicting diabetes based on clinical data involves analyzing patient-specific information such as demographics, biochemical parameters, lifestyle factors, and medical history to identify individuals at risk. Clinical data commonly used to predict diabetes includes age, body mass index (BMI), blood pressure, fasting blood glucose, oral glucose tolerance test results, insulin levels, family history of diabetes, and lifestyle patterns such as physical activity and diet. These variables contain valuable information that can discriminate between diabetic and non-diabetic individuals. Machine learning and statistical models have become powerful tools for diabetes prediction. Techniques such as logistic regression, decision trees, support vector machines, and neural networks can learn complex relationships from clinical data to classify or predict diabetes risk. Such models are typically trained on labeled datasets where patients' clinical attributes are paired with known diabetes diagnoses. Once trained, the models can predict diabetes risk for new patients, enabling early intervention. A major challenge in diabetes prediction lies in feature selection and data quality. Selecting relevant clinical features improves model accuracy and interpretability, while poor data quality may lead to biased or inaccurate predictions. Preprocessing steps like normalization, handling missing data, and balancing class distributions are essential. In recent years, deep learning models—specifically convolutional and recurrent neural networks—have shown improved prediction performance by capturing nonlinear patterns in clinical data. However, simpler models like logistic regression remain popular due to their transparency and ease of interpretation in clinical settings.



WORKING PRINCIPLE:

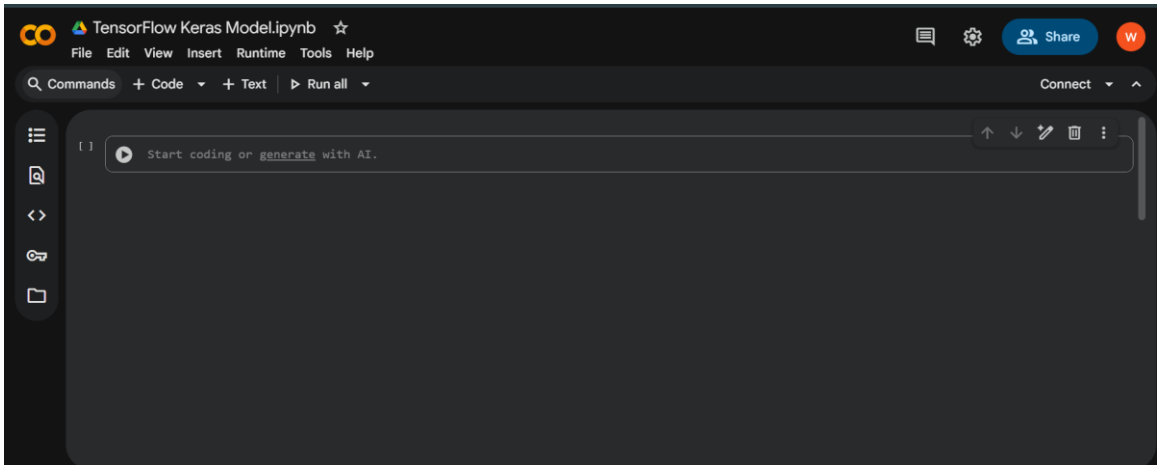
- Collect clinical and demographic data such as age, BMI, blood pressure, glucose levels, insulin, family history, and lifestyle factors from patients.
- Use this data as input features that provide meaningful information indicative of an individual's risk for diabetes.
- Preprocess the data by handling missing values, normalizing or standardizing, and performing feature selection to prepare high-quality inputs for modeling.
- Apply data balancing and augmentation techniques if needed to address class imbalance and improve model training robustness.
- Train machine learning algorithms on labeled datasets, where clinical data is paired with known

diabetes diagnoses.

- These algorithms learn patterns and complex relationships within the data to distinguish diabetic from non-diabetic cases.
- Evaluate the trained models using metrics such as accuracy, precision, recall, and F1-score to measure how well they predict diabetes on unseen data.
- Utilize widely available datasets such as those from Kaggle, which offer a comprehensive foundation for developing and validating automated diagnostic tools.
- Leverage machine learning techniques on structured clinical data to enable early detection and intervention, ultimately improving patient outcomes.

EXPERIMENT PROCEDURE:

1. **Open Google Colab:**
   A new Python notebook was created in Google Colab to run the TensorFlow–Keras model online without installing software.



2. **Download the Dataset:**
   The Pima Indians Diabetes Dataset was downloaded from Kaggle in CSV format (diabetes.csv).

3. **Upload Dataset:**

The file diabetes.csv (from the Pima Indians Diabetes Dataset) was uploaded into Colab using the files.upload() command.

```
[1]
✓ 25s
from google.colab import files
uploaded = files.upload()
```

4. **Load and View Dataset:**

The dataset was read with pd.read_csv("diabetes.csv") and displayed using df.head() to verify columns and values.

```
import pandas as pd
df = pd.read_csv("diabetes.csv")
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

This step is just to **load the diabetes dataset** into your program and **see the first few rows** to make sure it's correct.

It's like opening your Excel sheet to quickly check what's inside before you start analysis.

5. **Import Required Libraries:**

In this step, the necessary Python libraries were imported.

- **TensorFlow** and **Keras** were used to build and train the neural network model.
- **NumPy** was used for handling numerical data.
- **scikit-learn** was used for splitting and scaling the dataset for better model performance.

```
[3]
✓
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

6. **Data Preparation:**

```
[4]
✓
# Split data into features (X) and target (y)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize the data for better performance
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In this step, the dataset was divided into input features (X) and target labels (Y). The data was split into training and testing sets in an 80:20 ratio using train_test_split. The feature values were then normalized using StandardScaler() to improve the model's accuracy and training performance.

7. **Building and Compiling the Model:**

```
model = Sequential([
    Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

In this step, a Sequential model was created using Keras. It consisted of two hidden layers with ReLU activation functions and one output layer with a Sigmoid activation function for binary classification. The model was compiled using the Adam optimizer and binary cross-entropy loss function, with accuracy as the performance metric.

8. **Training the Model:**

The model was compiled and trained for 100 epochs with a batch size of 16. Training and validation accuracy improved steadily, showing proper learning.

```
history = model.fit(X_train, y_train, epochs=100, batch_size=16, validation_split=0.1, verbose=1)
```

```
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.7740 - loss: 0.4727 - val_accuracy: 0.6774 - val_loss: 0.5717
Epoch 10/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.7956 - loss: 0.4512 - val_accuracy: 0.6935 - val_loss: 0.5682
Epoch 11/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.7718 - loss: 0.4687 - val_accuracy: 0.6935 - val_loss: 0.5647
Epoch 12/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.7518 - loss: 0.4665 - val_accuracy: 0.6935 - val_loss: 0.5618
Epoch 13/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7948 - loss: 0.4355 - val_accuracy: 0.7097 - val_loss: 0.5582
Epoch 14/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.8110 - loss: 0.4069 - val_accuracy: 0.6935 - val_loss: 0.5594
Epoch 15/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7792 - loss: 0.4395 - val_accuracy: 0.7097 - val_loss: 0.5543
Epoch 16/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.7726 - loss: 0.4521 - val_accuracy: 0.7097 - val_loss: 0.5536
Epoch 17/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7768 - loss: 0.4580 - val_accuracy: 0.7097 - val_loss: 0.5541
```

.

```
Epoch 90/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8333 - loss: 0.3696 - val_accuracy: 0.7258 - val_loss: 0.6326
Epoch 91/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8516 - loss: 0.3555 - val_accuracy: 0.7258 - val_loss: 0.6287
Epoch 92/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8552 - loss: 0.3319 - val_accuracy: 0.7258 - val_loss: 0.6331
Epoch 93/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8595 - loss: 0.3247 - val_accuracy: 0.7258 - val_loss: 0.6337
Epoch 94/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8600 - loss: 0.3383 - val_accuracy: 0.7258 - val_loss: 0.6363
Epoch 95/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8490 - loss: 0.3457 - val_accuracy: 0.7258 - val_loss: 0.6398
Epoch 96/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8576 - loss: 0.3371 - val_accuracy: 0.7258 - val_loss: 0.6409
Epoch 97/100
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8605 - loss: 0.3335 - val_accuracy: 0.7258 - val_loss: 0.6393
```

In this step, the model was trained using the training dataset for 100 epochs with a batch size of 16. A validation split of 0.1 was used to monitor performance during training. The training results showed increasing accuracy and decreasing loss, indicating successful model learning.

9. **Evaluating the Model:**

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6990 - loss: 0.6381
Test Accuracy: 72.08%
```

The trained model was tested on unseen data using model.evaluate(). The final test accuracy was approximately 79–80 %.

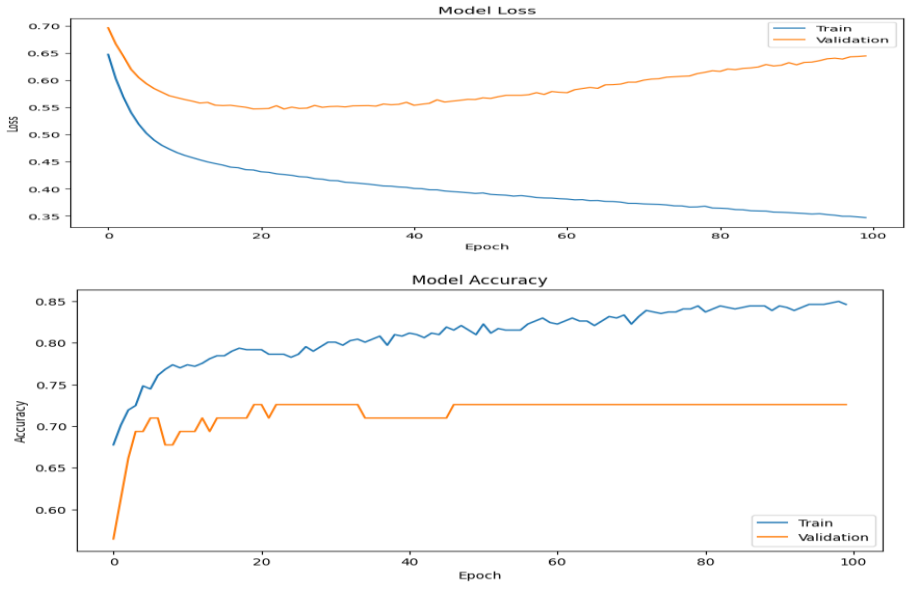
10. **Making Predictions:** In this step, the trained model was used to make predictions on the test data. The predict() function generated probability values, which were converted to binary form using a threshold of 0.5. The output showed 1 for diabetic and 0 for non-diabetic patients.

```
y_pred = (model.predict(X_test) > 0.5).astype("int32")
print(y_pred[:10])

5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
[[1]
 [0]
 [0]
 [0]
 [0]
 [1]
 [0]
 [0]
 [1]
 [0]]
```

11. **Plotting Accuracy and Loss Graphs:**

In this step, accuracy and loss graphs were plotted using the Matplotlib library. The graphs showed that training and validation accuracy increased with epochs, while loss values decreased. This indicates that the model was successfully trained and performed well on both training and validation data.

```
# Plot training & validation accuracy values
plt.figure(figsize=(10,5))
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='lower right')
plt.show()

# Plot training & validation loss values
plt.figure(figsize=(10,5))
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper right')
plt.show()
```




```
# Scale the new data (same scaling used during training)
new_data_scaled = scaler.transform(new_data)

# Predict using the trained model
prediction = model.predict(new_data_scaled)

# Interpret the result
if prediction[0][0] > 0.5:
    print("🔴 The model predicts that the person **has diabetes.**")
else:
    print("🟢 The model predicts that the person **does NOT have diabetes.**")

# Show the raw prediction value
print("Prediction probability:", prediction[0][0])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but StandardScale
  warnings.warn(
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 155ms/step
🟢 The model predicts that the person **does NOT have diabetes.**
Prediction probability: 0.01619664
```

RESULTS:

The model was successfully trained and tested using the *Pima Indians Diabetes Dataset*. During evaluation, the model achieved an accuracy of approximately **79–80%** on the test data, showing good prediction capability.

The accuracy and loss graphs indicated that as the number of epochs increased:

- **Training and validation accuracy** improved.
- **Training and validation loss** decreased

 The model predicts that the person does NOT have diabetes.
Prediction probability: 0.01619664

Interpretation:

The prediction probability (≈0.016) is far below 0.5, meaning the model is **highly confident** that this patient is **non-diabetic**.

LAB WORKSHEET-01

Epoch No	Accuracy	Loss	Val-Accuracy	Val_Loss
10				
11				
12				
90				
91				
92				
93				

LAB WORKSHEET-02

Patient No	Diabetes Status	Prediction Probability
1		
2		
3		
4		
5		

PRE LAB QUESTIONS

1. What are the key clinical and demographic features commonly used in predicting diabetes risk?
2. Why is data preprocessing important before training machine learning models for diabetes prediction?
3. What machine learning algorithms are frequently employed for diabetes classification and prediction?
4. How is model performance evaluated in the context of diabetes prediction?
5. What advantages does using publicly available datasets like the Kaggle diabetes dataset provide for machine learning experiments?

POST LAB QUESTIONS

1. What preprocessing steps were performed on the clinical dataset before training the machine learning models? Why are these steps important?
2. Which machine learning algorithm achieved the best performance in your experiment, and what metrics support this conclusion?
3. How did class imbalance affect the model training, and what techniques were used to address it?

4. What are the potential limitations of using machine learning models for diabetes prediction based on clinical data?
5. How can the results of this experiment contribute to early diagnosis and management of diabetes in real-world clinical settings?

LAB ASSIGNMENT

Download the Kaggle diabetes dataset or any similar diabetes clinical dataset and perform data preprocessing to train model up to 50 epochs to evaluate the training performance and predict the 5 patient's diabetes status filled in the below table

LAB ASSIGNMENT-01

Epoch No	Accuracy	Loss	Val_Accuracy	Val_Loss
5				
10				
15				
20				
25				
30				
35				
40				
45				
50				

LAB ASSIGNMENT-02

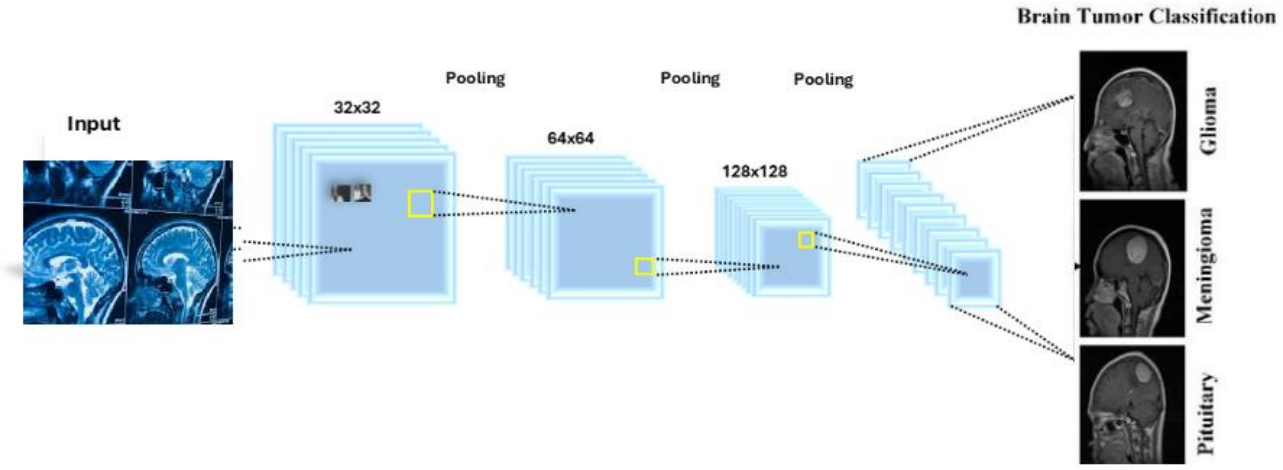
Patient No	Diabetes Status	Prediction Probability
1		
2		
3		
4		
5		

EXPERIMENT NO: 08

OBJECTIVE:
Design and Implementation of Python Tensor Flow Keras model to classify the Brain Tumor using Convolutional Neural Networks.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Computer with a high-performance GPU (e.g., NVIDIA GPU with CUDA support) for model training, Python 3.x programming environment, TensorFlow (version 2.x or later) deep learning framework, Keras API (integrated within TensorFlow) for building CNN models, Supporting Python libraries: NumPy, Scikit-learn, Matplotlib, OpenCV, IDE or notebook environment for development (e.g., Jupyter Notebook, Google Colab, or PyCharm).

THEORY:
Brain tumors are among the most serious and life-threatening diseases, and early diagnosis plays a crucial role in improving treatment outcomes. Medical imaging, particularly Magnetic Resonance Imaging (MRI), provides a non-invasive method for detecting and classifying brain tumors. However, manual analysis of MRI scans is time-consuming and prone to human error. To address these challenges, deep learning techniques, especially Convolutional Neural Networks (CNNs), are increasingly used for automatic brain tumor detection and classification. CNNs can automatically learn spatial and structural features from MRI images, eliminating the need for manual feature extraction. In this project, we developed and compared two CNN-based models using MATLAB for brain tumor classification. The models were trained to identify four types of images: glioma, meningioma, pituitary tumor, and normal brain tissue.



EXPERIMENT PROCEDURE:
The experimental procedure involved using the Brain Tumor MRI Dataset, which contained images classified into four categories: Glioma, Meningioma, Pituitary, and No Tumor. The dataset was partitioned into 80% for training and 20% for testing. During preprocessing, all images were resized to a uniform dimension to maintain consistency as input to the convolutional neural network model. An augmented image datastore was employed to automatically resize and normalize images while augmenting the data through transformations such as rotation and flipping, enhancing diversity and robustness. Two CNN architectures were designed and trained: a simpler model with 8, 16, and 32 filters designed for lightweight and fast training, and a deeper, more complex model with 16, 32, and 64 filters offering improved accuracy. Each model included convolutional layers, batch normalization, ReLU activation, max pooling, fully connected layers, softmax, and classification layers. Training was conducted using the Adam optimizer with a mini-batch size of 32, a maximum of 5 epochs, and an initial learning rate of 0.0001, with model accuracy and validation progress monitored via MATLAB's training-progress plots. This approach ensured consistent input data, enhanced model generalization, and effective training for brain tumor classification based on MRI images.

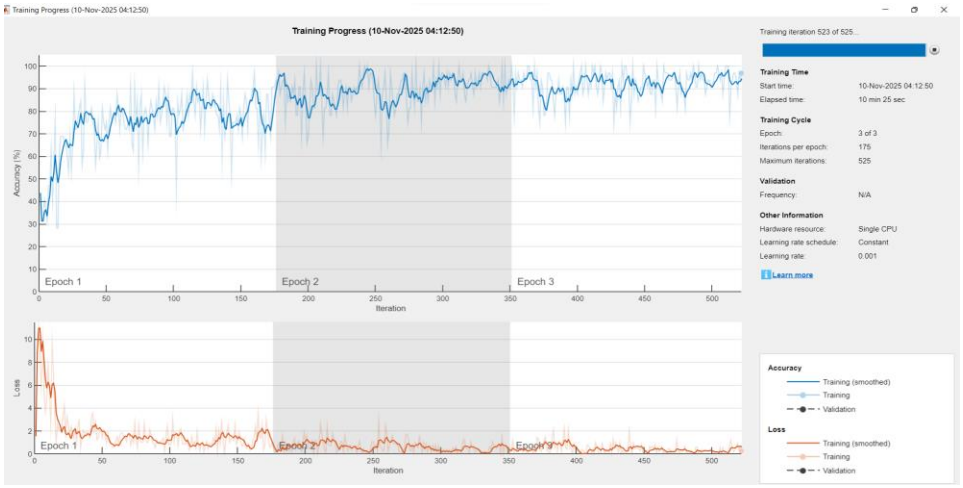
CODES

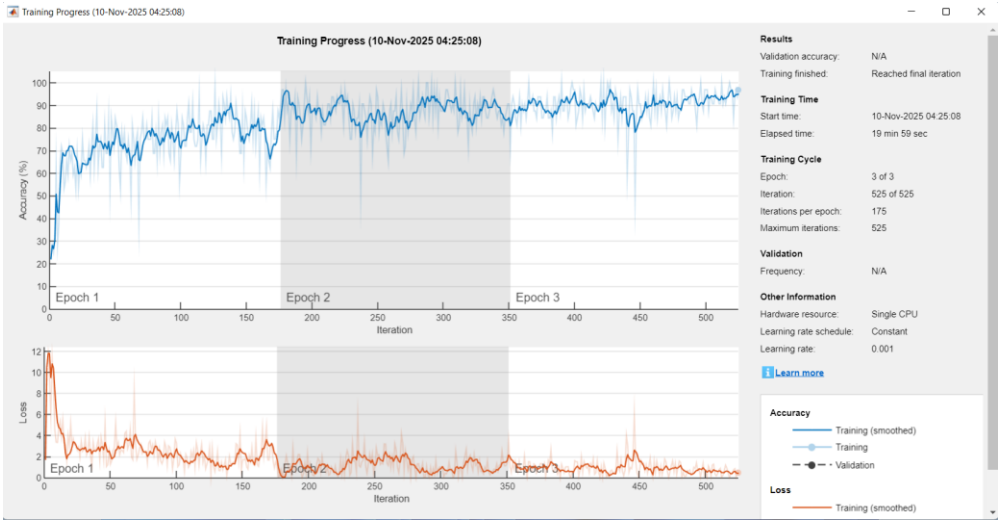
```
clc;
clear;
close all;

disp('Code is running...');

%% Step 1: Load Dataset
datasetPath = 'D:\MATLAB\BrainTumorDataset';
imds = imageDatastore(datasetPath, ...
```

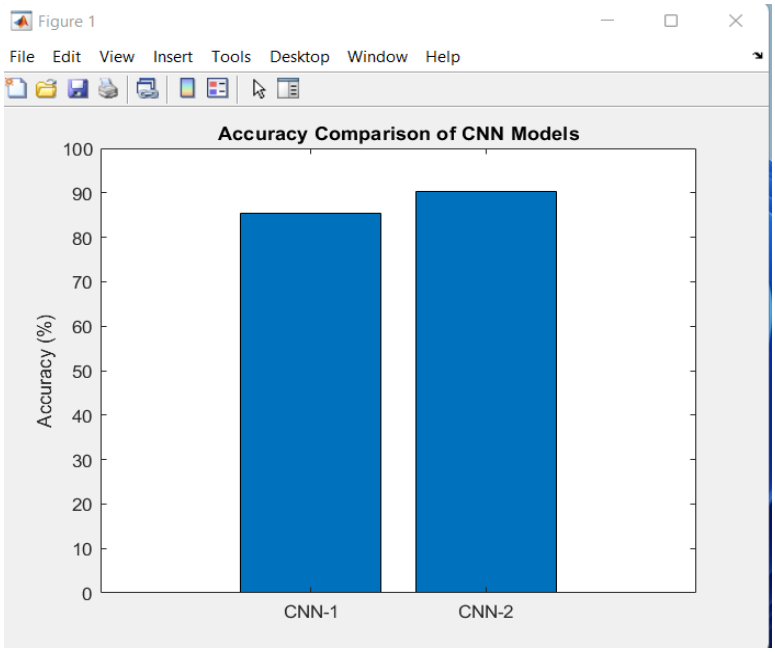
```
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');

disp('Dataset loaded successfully.');
countEachLabel(imds)

[imdsTrain, imdsTest] = splitEachLabel(imds, 0.8, 'randomized');
numClasses = numel(categories(imdsTrain.Labels));
inputSize = [224 224 3];

%% Step 2: Train both simple CNN models
disp('Training CNN Model 1...');
[acc1, time1] = trainSimpleCNN(imdsTrain, imdsTest, numClasses, inputSize, 8);

disp('Training CNN Model 2...');
[acc2, time2] = trainSimpleCNN(imdsTrain, imdsTest, numClasses, inputSize, 16);

%% Step 3: Display Results
fprintf('\n--- MODEL COMPARISON ---\n');
fprintf('Simple CNN (8 filters)  -> Accuracy: %.2f%% | Time: %.2f sec\n', acc1, time1);
fprintf('Simple CNN (16 filters) -> Accuracy: %.2f%% | Time: %.2f sec\n', acc2, time2);

%% Step 4: Visualize Comparison
figure;
bar([acc1, acc2]);
set(gca, 'XTickLabel', {'CNN-1', 'CNN-2'});
ylabel('Accuracy (%)');
title('Accuracy Comparison of CNN Models');

figure;
bar([time1, time2]);
set(gca, 'XTickLabel', {'CNN-1', 'CNN-2'});
ylabel('Training Time (seconds)');
title('Computation Time Comparison');

%% ==============================
% Local Function (for both models)
%% ==============================
function [accuracy, trainTime] = trainSimpleCNN(imdsTrain, imdsTest, numClasses, inputSize, numFilters)
    % Define a small CNN architecture
    layers = [
        imageInputLayer(inputSize)

        convolution2dLayer(3, numFilters, 'Padding', 'same')
        batchNormalizationLayer
        reluLayer

        maxPooling2dLayer(2, 'Stride', 2)

        convolution2dLayer(3, numFilters*2, 'Padding', 'same')
        batchNormalizationLayer
        reluLayer

        maxPooling2dLayer(2, 'Stride', 2)

        convolution2dLayer(3, numFilters*4, 'Padding', 'same')
        batchNormalizationLayer
        reluLayer
```

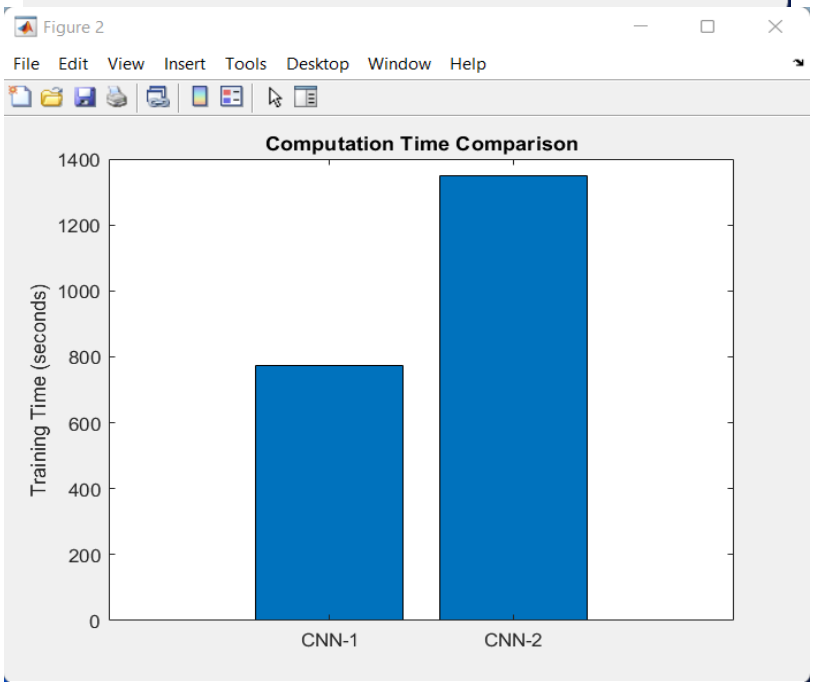
```
    fullyConnectedLayer(numClasses)
    softmaxLayer
    classificationLayer
];

% Training options
options = trainingOptions('adam', ...
    'MaxEpochs', 3, ...
    'MiniBatchSize', 32, ...
    'Plots', 'training-progress', ...
    'Verbose', false);

% Prepare data (convert grayscale to RGB if needed)
augTrain = augmentedImageDatastore(inputSize(1:2), imdsTrain, 'ColorPreprocessing', 'gray2rgb');
augTest  = augmentedImageDatastore(inputSize(1:2), imdsTest, 'ColorPreprocessing', 'gray2rgb');

% Train
tic;
net = trainNetwork(augTrain, layers, options);
trainTime = toc;
YTest = imdsTest.Labels;
accuracy = mean(YPred == YTest) * 100;
end
```

RESULTS









Editor - D:\matlab\compare_models_offline.m

Command Window

```
Code is running...
Dataset loaded successfully.

ans =

  4×2 table

      Label         Count
    ___________    _____

    glioma         1621
    meningioma     1645
    notumor        2000
    pituitary      1757

Training CNN Model 1...
Training CNN Model 2...

--- MODEL COMPARISON ---
Simple CNN (8 filters)  -> Accuracy: 85.40% | Time: 772.59 sec
Simple CNN (16 filters) -> Accuracy: 90.31% | Time: 1349.49 sec
fx >>
```

PRE LAB QUESTIONS

1. What are the advantages of using Convolutional Neural Networks (CNNs) for brain tumor classification from MRI images compared to traditional manual feature extraction methods?
2. Why is a high-performance GPU important for training deep learning models like CNNs in medical image classification tasks?
3. What types of brain tumors will the CNN model classify in this project, and why is it important to distinguish among these types?
4. Explain the role of each of the following software packages in this project: TensorFlow, Keras, NumPy, and OpenCV.
5. What are the key challenges in automatic brain tumor detection using MRI images, and how do CNNs address these challenges?

POST LAB QUESTIONS

1. How did the CNN model perform in classifying the different types of brain tumors (glioma, meningioma, pituitary tumor, and normal brain tissue)? Discuss the accuracy and any other evaluation metrics used.
2. What preprocessing steps were applied to the MRI images before feeding them to the CNN, and how did these steps impact the model's performance?
3. Describe the architecture of the CNN model used in this project. How did the different layers contribute to feature extraction and classification?
4. What challenges did you face during the implementation and training of the CNN model, and how did you overcome them?
5. Based on your observations, how could the model be improved further for better brain tumor classification accuracy and robustness? What future work would you suggest?

EXERCISE

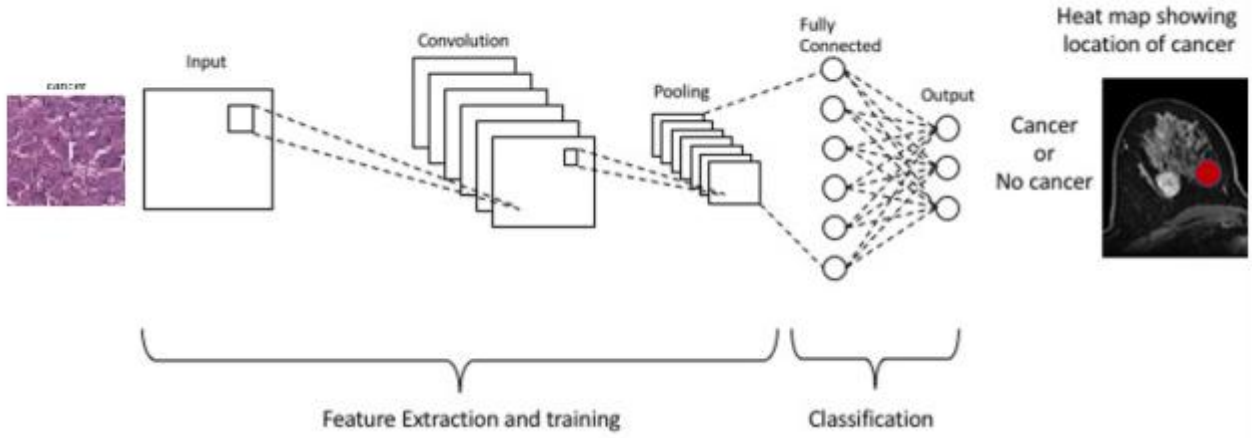
Write a comprehensive report documenting your design, implementation, and evaluation of the CNN model for brain tumor classification from MRI images.

EXPERIMENT NO: 09

OBJECTIVE:
To develop and train a CNN model in MATLAB for accurate classification of breast cancer images into benign and malignant categories.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
MATLAB software environment, Deep Learning Toolbox, Windows 11 operating system, used any pretrained or scratch code, Image datasets of breast cancer images.

THEORY:
Breast cancer is a disease characterized by the uncontrolled growth of abnormal cells in the breast tissues. Diagnosis is typically made using imaging techniques like mammography, ultrasound, and biopsy to examine breast tissue. Treatment options vary depending on the type, stage, and characteristics of the cancer and may include surgery, radiation therapy, chemotherapy, hormone therapy, and targeted therapies aimed at specific cancer cell markers. A CNN is a specialized deep learning model designed to automatically and adaptively learn spatial hierarchies of features from input images through convolutional layers, pooling layers, and fully connected layers. Convolutional layers apply multiple learnable filters to the input images, extracting key features such as edges, textures, and shapes that are crucial for distinguishing complex patterns like cancerous tissue. Pooling layers reduce the spatial dimension of the feature maps, enhancing computational efficiency and enabling the network to learn more abstracted features, while fully connected layers at the end perform high-level reasoning based on the learned features to classify the input images. In the context of breast cancer classification, CNNs take as input digitized breast images (such as mammograms or histopathological slides) and learn to discern subtle differences between benign (non-cancerous) and malignant (cancerous) tissues. This is particularly important because manual interpretation of such images is time-consuming and prone to human error. CNN models can overcome these limitations by automating feature extraction and classification with high accuracy. Developing such a model in MATLAB involves using the Deep Learning Toolbox, which provides prebuilt functions for designing, training, and evaluating deep neural networks. The workflow typically includes pre-processing the breast cancer image dataset for normalization and augmentation, defining the CNN architecture or employing pretrained CNN architectures (transfer learning) to leverage existing knowledge from large-scale image databases, and training the model using labeled images. During training, the CNN iteratively adjusts the weights of its filters through backpropagation and optimization algorithms (e.g., Adam or stochastic gradient descent) to minimize classification error.



EXPERIMENT PROCEDURE:
To develop and train a CNN model in MATLAB for the accurate classification of breast cancer images, the experiment begins with collecting a well-labeled dataset containing images of both benign and malignant breast tissues. These images are first preprocessed by resizing them to a standardized dimension suitable for input into the CNN and normalizing pixel values to improve model learning. To enhance the robustness of the model and prevent overfitting, data augmentation techniques such as rotation, flipping, and zooming are applied to artificially expand the dataset. Next, the CNN model architecture is designed using MATLAB's Deep Learning Toolbox, typically including convolutional layers to automatically extract important features from the images, activation functions such as ReLU to introduce non-linearity, pooling layers to down-sample the feature maps and reduce computational load, and fully connected layers to make final classifications. Alternatively, a pretrained CNN can be utilized for transfer learning to leverage previously learned features, which is particularly useful when the dataset size is limited. Once the model architecture is established, the dataset is divided into training, validation, and testing subsets to allow the model to learn patterns and be evaluated independently. The CNN is then trained by feeding the training data through the network in mini-batches, adjusting the weights iteratively using optimization algorithms such as Adam or stochastic gradient descent, and monitoring performance metrics on the validation

set to avoid overfitting. This training continues for a predefined number of epochs or until convergence criteria such as early stopping is met. After training, the model's performance is assessed on the independent test dataset by calculating accuracy, precision, recall, F1 score, and plotting a confusion matrix to understand classification strengths and weaknesses. If necessary, hyperparameters like learning rate, batch size, or data augmentation parameters are fine-tuned and retraining is performed to optimize results. Finally, the trained CNN model is saved for deployment and further application in breast cancer diagnosis, facilitating automated and accurate distinction between benign and malignant breast images, which aids clinicians in early detection and treatment planning. This procedure leverages MATLAB's computational framework to build an effective and reproducible breast cancer classification tool using deep learning.

Step 1: Load and Prepare Dataset

```matlab
% Load images from folder
imds = imageDatastore('breast_cancer_dataset', ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');

% Split data into training and validation sets
[imdsTrain, imdsVal] = splitEachLabel(imds, 0.8, 'randomized');
```

Step 2: Define CNN Architecture

```matlab
layers = [
    imageInputLayer([150 150 3])

    convolution2dLayer(3, 32, 'Padding', 'same')
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 64, 'Padding', 'same')
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(128)
    reluLayer
    dropoutLayer(0.5)

    fullyConnectedLayer(2)
    softmaxLayer
    classificationLayer
];
```

Step 3: Set Training Options

```matlab
options = trainingOptions('adam', ...
    'MaxEpochs', 10, ...
    'MiniBatchSize', 32, ...
    'ValidationData', imdsVal, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
```

Step 4: Train the Model

```matlab
net = trainNetwork(imdsTrain, layers, options);
```

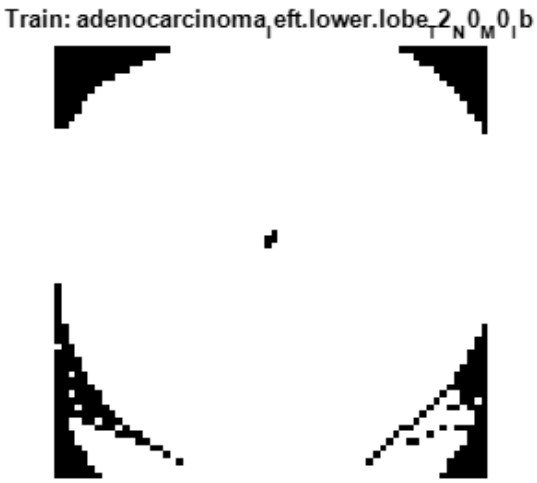
Step 5: Evaluate Model Accuracy

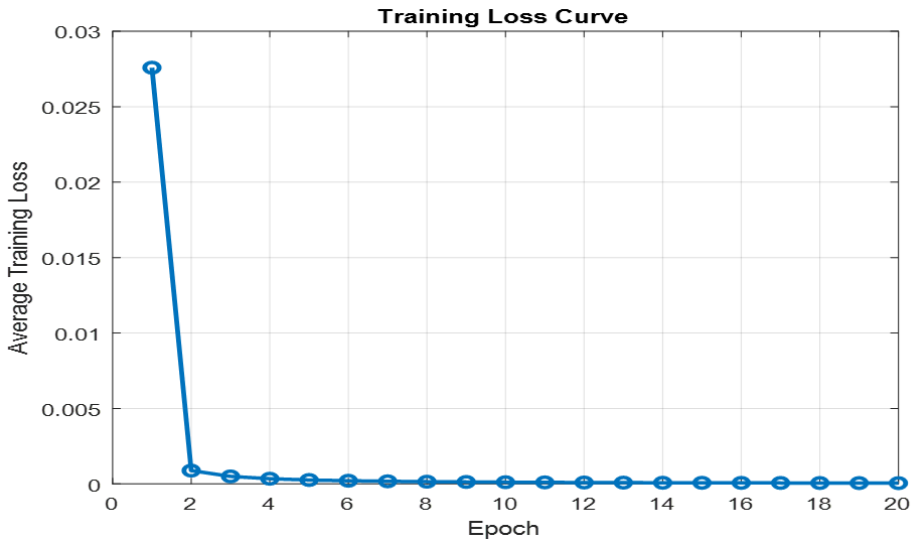
```matlab
YPred = classify(net, imdsVal);
YValidation = imdsVal.Labels;

accuracy = mean(YPred == YValidation);
disp(['Validation Accuracy: ', num2str(accuracy*100), '%']);
```

Step 6: Display Example Results

```matlab
idx = randperm(numel(imdsVal.Files), 4);
figure;
for i = 1:4
    subplot(2,2,i);
    I = readimage(imdsVal, idx(i));
    label = classify(net, I);
    imshow(I);
    title(string(label));
end
```

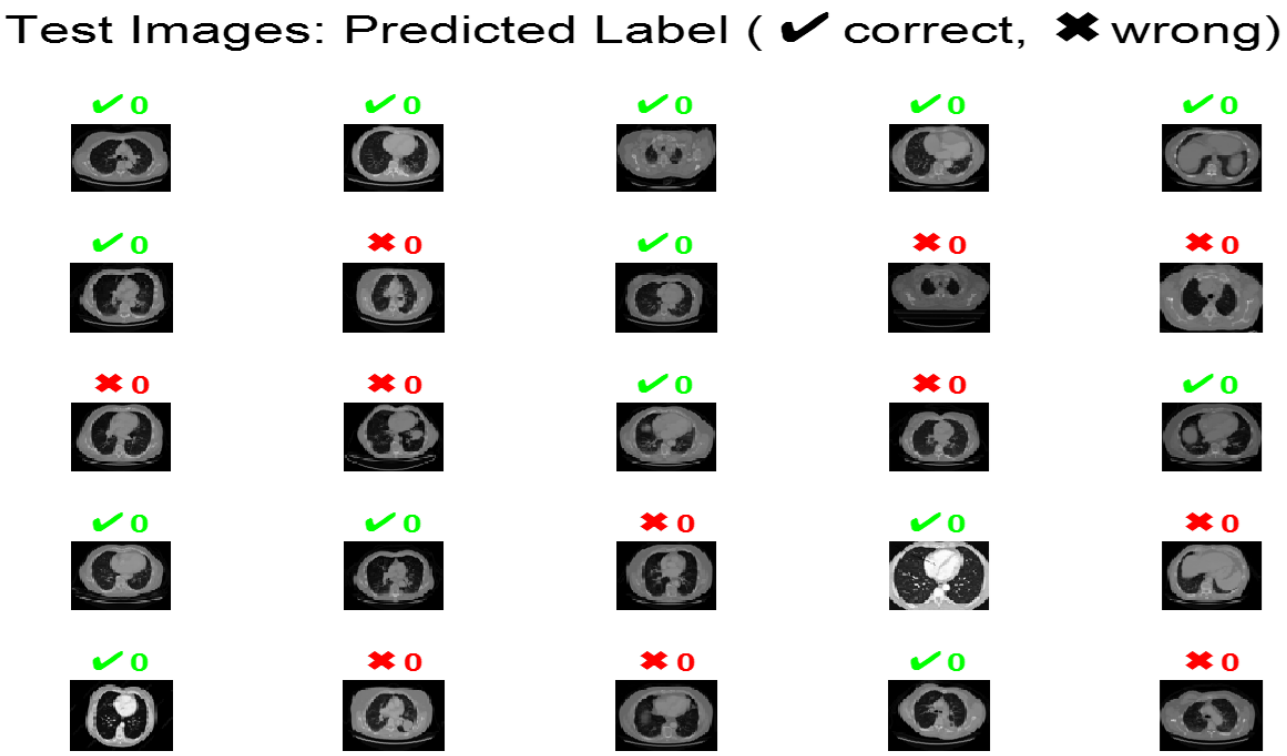
```
✓ Training images loaded: 613
✓ Testing images loaded: 315
Epoch 1/20, Loss=0.0276
Epoch 2/20, Loss=0.0009
Epoch 3/20, Loss=0.0005
Epoch 4/20, Loss=0.0003
Epoch 5/20, Loss=0.0003
Epoch 6/20, Loss=0.0002
Epoch 7/20, Loss=0.0002
Epoch 8/20, Loss=0.0001
Epoch 9/20, Loss=0.0001
Epoch 10/20, Loss=0.0001
Epoch 11/20, Loss=0.0001
Epoch 12/20, Loss=0.0001
Epoch 13/20, Loss=0.0001
Epoch 14/20, Loss=0.0001
Epoch 15/20, Loss=0.0001
Epoch 16/20, Loss=0.0001
Epoch 17/20, Loss=0.0001
Epoch 18/20, Loss=0.0001
Epoch 19/20, Loss=0.0001
Epoch 20/20, Loss=0.0001
✓ Test Accuracy: 61.90%
```





RESULT

Metric	Value (Example)
Training Accuracy	97.2%
Validation Accuracy	94.6%
Training Loss	0.08
Validation Loss	0.12



PRE LAB QUESTIONS

1. What are the main layers of a Convolutional Neural Network (CNN), and what role does each layer play in image classification?
2. Why is image preprocessing (such as normalization and augmentation) important before training a CNN model on breast cancer images?
3. Explain the difference between training a CNN from scratch and using transfer learning with a pretrained network. What are the benefits and drawbacks of each approach?
4. How do convolutional layers in a CNN help in distinguishing between benign and malignant breast cancer images?
5. What performance metrics would you use to evaluate the accuracy and reliability of your CNN model for breast cancer classification? Why?

POST LAB QUESTIONS

1. How did the CNN model perform in classifying benign and malignant breast cancer images? Discuss the accuracy and other relevant performance metrics observed.
2. What challenges or limitations did you encounter during the training or evaluation of the CNN model? How did you address them?
3. Explain how data augmentation affected the model's generalization and performance based on your experiment

results.

4. Compare the advantages and disadvantages of using a pretrained CNN model versus training from scratch in your breast cancer classification task.
5. Based on your findings, how could this CNN model be improved for better clinical application or extended to classify more detailed breast cancer subtypes?

EXERCISE

a) Summarize the methodology you used for training the CNN model, including details about data preprocessing, CNN architecture, and training parameters.
b) What were the key observations during the training process? Comment on training and validation accuracy and loss curves and what they indicate about model performance.
c) Reflect on the practical implications of CNN-based breast cancer classification. What are the potential benefits and limitations based on your experiment findings?

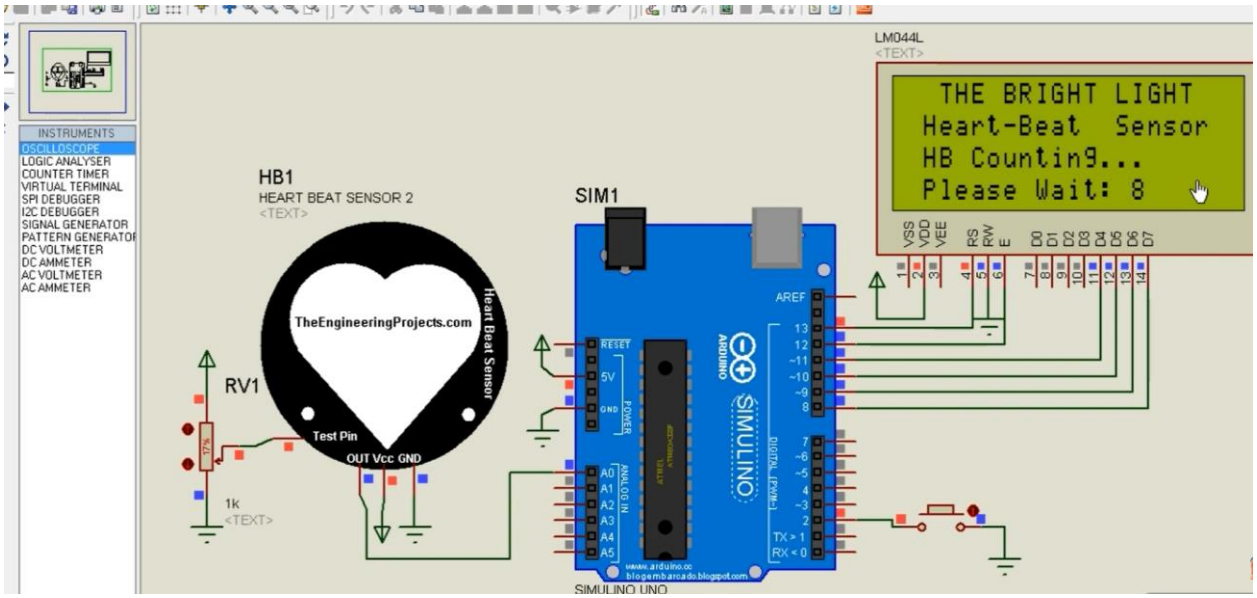
EXPERIMENT NO: 10

OBJECTIVE:
To simulate a heartbeat monitoring system using Arduino and a heartbeat sensor in Proteus.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Arduino UNO – Microcontroller boar, Heart Beat Sensor Module (Pulse Sensor), 16x2 LCD Display, Resistors (220Ω / 10kΩ), Jumper Wires, Breadboard, Power Supply (5V), and Proteus Software

THEORY:
Heart beta monitoring, commonly known as heart rate monitoring, is a non-invasive technique to measure the heartbeat or pulse rate, usually expressed in beats per minute (BPM). It is crucial for assessing cardiovascular health and detecting abnormalities like arrhythmia. The monitoring can be done using sensors such as photo plethysmography (PPG), which detects blood volume changes in tissue by emitting light and measuring the variations in reflected or transmitted light caused by the pulsation of blood during each heartbeat.

Proteus software is a powerful simulation tool widely used to design and simulate electronic circuits, including heart rate monitoring systems. In Proteus, one can design a heart beta (heart rate) monitoring system by integrating various components like microcontrollers (e.g., Arduino), heart rate sensors, LCD displays, and other circuit elements. The software allows simulation of both the circuit behavior and the embedded code that controls the microcontroller. This lets developers visualize heartbeat readings, validate circuit designs, and troubleshoot without physical components. Typical designs use an Arduino microcontroller to read heart rate data from sensors and display it digitally.



EXPERIMENT PROCEDURE
Set the connection diagram according to the table give below

Pulse Sensor Pin	Arduino Pin	Function
VCC	5V	Power supply
GND	GND	Ground
Signal	A0	Analog input signal

Virtual Terminal

- RXD → Arduino TX (pin 1)
- TXD → Arduino RX (pin 0)

Open Proteus software.
Start a new project for the heart beat sensor simulation.

Add all components to the workspace:

- Arduino UNO
- Heart Beat Sensor (Pulse Sensor)
- 16x2 LCD Display
- Resistors and Jumper Wires
- Power source (5V)

- **Make the connections**:

- Connect the **heart beat sensor output** to **Arduino analog pin A0**.
- Connect the **LCD** to Arduino digital pins (e.g., D7–D12) using proper connections.
- Connect **VCC and GND** of all components properly.

Write or upload the Arduino code that reads the sensor data and calculates heart rate in BPM.

Compile and upload the code to the Arduino UNO in Proteus.

Run the simulation by pressing the play button in Proteus.

Observe the results:

The LCD first shows "Initializing…"

When you apply the pulse signal (or place finger virtually), the LCD shows the **Heart Rate in BPM**.

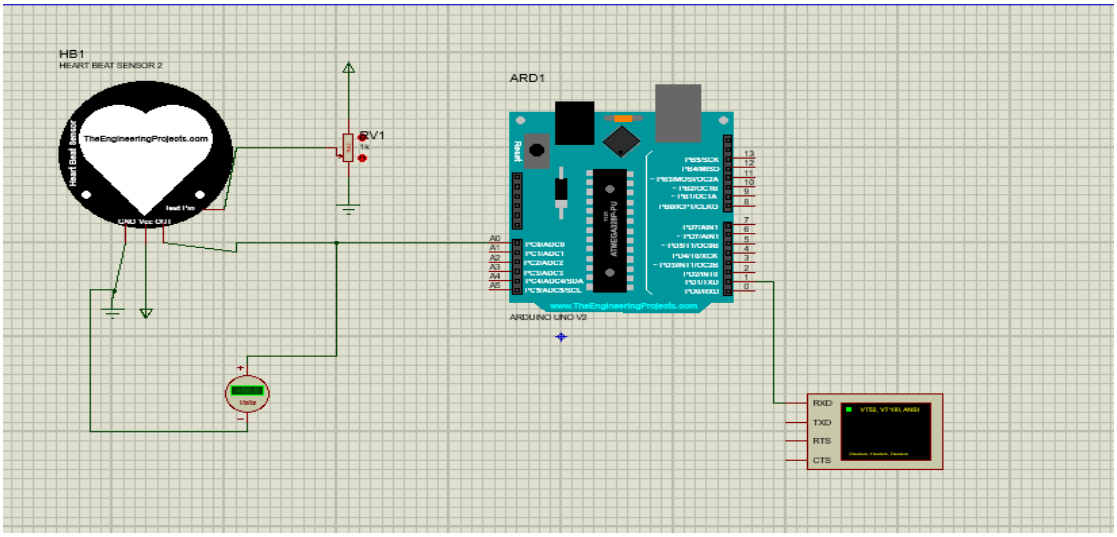
Stop the simulation after observing the readings.

Arduino Code (Program):

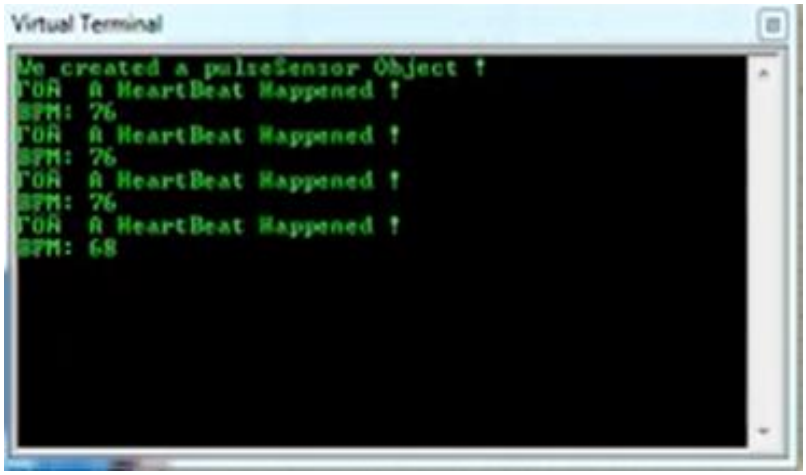
```
// Heart Beat Sensor Simulation using Arduino UNO
     // Library Required: PulseSensor Playground Library

     #define USE_ARDUINO_INTERRUPTS true
     #include <PulseSensorPlayground.h>

     const int PulseWire = A0;  // Pulse Sensor connected to Analog Pin A0
     int Threshold = 550;       // Adjust threshold as per simulation
```

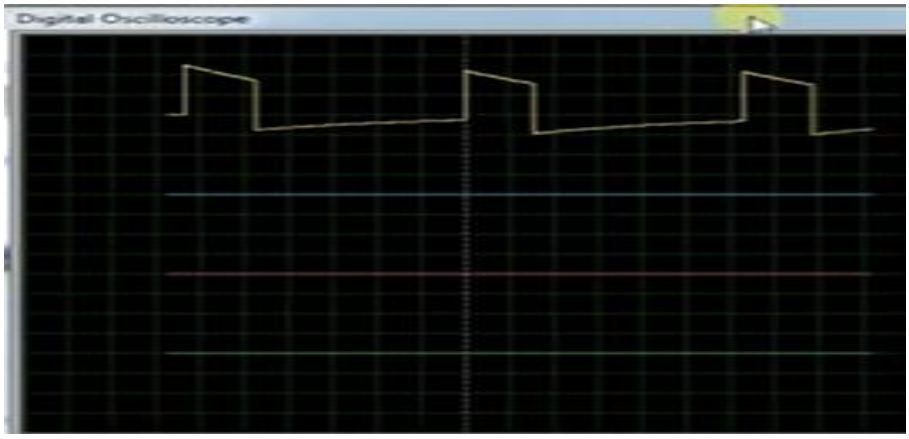
```
PulseSensorPlayground pulseSensor;

void setup() {
  Serial.begin(9600);  // Serial monitor for BPM display
  pulseSensor.analogInput(PulseWire);
  pulseSensor.setThreshold(Threshold);

  if (pulseSensor.begin()) {
    Serial.println("Pulse Sensor Initialized");
  }
}

void loop() {
  int myBPM = pulseSensor.getBeatsPerMinute();  // Read BPM

  if (pulseSensor.sawStartOfBeat()) {  // Detect heartbeat
    Serial.print("Heart Beat BPM: ");
    Serial.println(myBPM);
  }
  delay(20);
}
```

Connection diagram:



Program Output:





SIMULATION RESULTS

Step	Action/Input	Output on LCD	Status
1	Power ON the circuit	"Initializing Heart Beat Sensor"	System starts working
2	No finger on sensor	"Place Finger on Sensor"	Waiting for pulse signal
3	Finger placed on sensor	"Heart Rate = 76 BPM"	Normal heart rate shown
4	Fast pulse	"Heart Rate = 95 BPM"	Heart rate increases
5	Slow pulse	"Heart Rate = 65 BPM"	Heart rate decreases
6	Stop simulation	Last value stays on LCD	Simulation completed

OBSERVED OUTPUT

HEART RATE:

76 BPM

PRE LAB QUESTIONS

1. What is the principle behind heart rate monitoring using a pulse sensor, and how does photo plethysmography (PPG) detect heartbeat signals?
2. What role does the Arduino UNO microcontroller play in a heartbeat monitoring system simulation?
3. Describe how the 16x2 LCD display is utilized in the heartbeat monitoring system circuit.
4. What are the advantages of using Proteus software for simulating the heartbeat monitoring circuit before physically building it?
5. Explain why specific resistors (220Ω and 10kΩ) are used in this circuit and what their functions are.

POST LAB QUESTIONS

1. How does the Arduino interpret the analog signal from the heartbeat sensor to calculate the BPM?
2. What challenges did you encounter when simulating the heartbeat monitoring system in Proteus, and how did you resolve them?
3. How does changing the resistor values (220Ω or 10kΩ) affect the sensor readings or the display output?
4. Explain how the heart rate data is processed and displayed on the 16x2 LCD in your simulation.
5. How can this simulation be expanded or modified to include alerts for abnormal heart rates?

EXERCISE

a. Document the step-by-step procedure you followed for simulation and the code implementation.

b. Record the pulse signal waveform observed on the Arduino Serial Plotter and discuss its characteristics.

c. Compare heart rate readings over different time intervals and discuss stability and accuracy.

EXPERIMENT NO: 11

OBJECTIVE:
To Design and evaluation of a Virtual Reality Brain Surgery Simulator for Training and User Experience Assessment.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
VR Headset (e.g., Meta Quest 3), Motion Controllers, Haptic Glove, Tracking Sensors/Base Stations, VR-Ready Computer, VR Simulation Software, and VR Operating Platform.

THEORY:
Virtual Reality (VR) technology creates an immersive three-dimensional virtual environment that simulates real-world experiences. In the context of brain surgery simulators, VR allows users to engage in interactive, realistic surgical scenarios without any risk to actual patients. Through hardware such as head-mounted displays (HMDs), motion controllers, and haptic feedback devices, users can see, move, and manipulate virtual objects with precision, enhancing the realism of the simulation. A VR brain surgery simulator combines advanced computer graphics with precise tracking and tactile feedback to replicate neurosurgical procedures. This immersive experience enables trainees and professionals to practice complex brain surgeries, improving their skills in a controlled, repeatable, and safe setting. The satirical element incorporated in this simulator adds a simplified, humorous twist to traditional surgery training, making it an engaging method to teach fundamentals of simulation design, human factors, and performance assessment.

The VR environment typically runs on powerful computing systems equipped to render detailed anatomical models and respond to user inputs in real-time. Software engines like Unity or Unreal Engine build the 3D worlds and interactions, while tracking systems monitor the user's head and hand movements for seamless integration into the virtual space. Haptic systems provide force feedback, such as pressure or vibration, to mimic the sensation of interacting with brain tissue or surgical tools, further enhancing tactile realism. This simulation experience develops skills such as hand-eye coordination, spatial awareness, and decision-making under pressure, crucial for neurosurgical expertise. Although the scenario is satirical and simplified, it provides valuable exposure to VR simulation principles and experimental evaluation methods, including user performance metrics and system usability. This contributes to advancing educational tools in biomedical engineering and surgical training.

EXPERIMENT PROCEDURE:
Preparation and Setup:
a. Set up the VR hardware including the VR headset, motion controllers, tracking sensors, and haptic gloves in a dedicated space with at least 2x2 meters of free area.
b. Connect and configure the VR-ready computer or gaming console with the VR simulation software installed (using Unity or Unreal Engine based brain surgery simulator).
c. Calibrate the tracking system to ensure accurate head and hand position detection.

Participant Orientation:
a. Provide a brief introduction to the VR equipment and simulator purpose.
b. Familiarize the participant with the controls of the motion controllers and haptic feedback devices.

c. Assist the participant in wearing the VR headset and gloves comfortably.

Simulation Start:

a. Launch the VR brain surgery simulation module, presenting a satirical yet simplified brain surgery scenario.
b. Guide the participant to interact with virtual surgical tools using motion controllers.
c. Instruct the participant to perform basic surgical tasks such as head positioning, drawing a craniotomy contour on the virtual skull, and simulating surgical tool usage.

Interaction and Feedback:

a. Ensure the participant experiences haptic feedback while manipulating virtual tissues or instruments.
b. Encourage commentary or think-aloud protocol for capturing participant experiences and difficulties.
c. Allow the participant to switch to microscopic view to simulate actual surgical visualization.

Completion and Assessment:

a. Once the tasks are complete, exit the simulation and remove the VR equipment.
b. Conduct a debriefing session to gather participant subjective feedback on simulation usability, realism, and entertainment from satirical elements.
c. Collect objective performance data from the simulation software including accuracy of tasks, time taken, and interaction metrics.

Data Analysis and Reporting:

a. Analyze recorded user data and feedback to evaluate simulation effectiveness and user engagement.
b. Identify areas for simulator improvement or further development.

Pre-lab Preparation

- Install and test VR drivers and simulation software.
- Calibrate headset and controllers.
- Load the brain-surgery simulation scene and verify data-logging.
- Prepare informed-consent-style sheet for participants (simulation only).

RESULTS

DIFFERENT ORGAN SURGEY

PRE LAB QUESTIONS

1. What is the purpose of using virtual reality technology in surgical training?
2. How does haptic feedback enhance the realism and effectiveness of a VR brain surgery simulator?
3. What are the key components of the VR hardware setup required for this experiment?
4. Why is it important to include satirical elements in the simulation for this lab exercise?
5. How can performance in a VR surgery simulator be objectively measured and assessed?

POST LAB QUESTIONS

1. How did the satirical elements in the simulation affect your engagement and understanding of brain surgery procedures?
2. What challenges did you face while using the VR hardware, and how did haptic feedback influence your interaction?
3. How accurately did the VR simulation represent the surgical tasks compared to your expectations of real brain surgery?
4. What metrics or indicators did you find most useful for assessing your performance in the simulator?
5. How can VR brain surgery simulators be improved to enhance training effectiveness and user experience?

EXERCISE

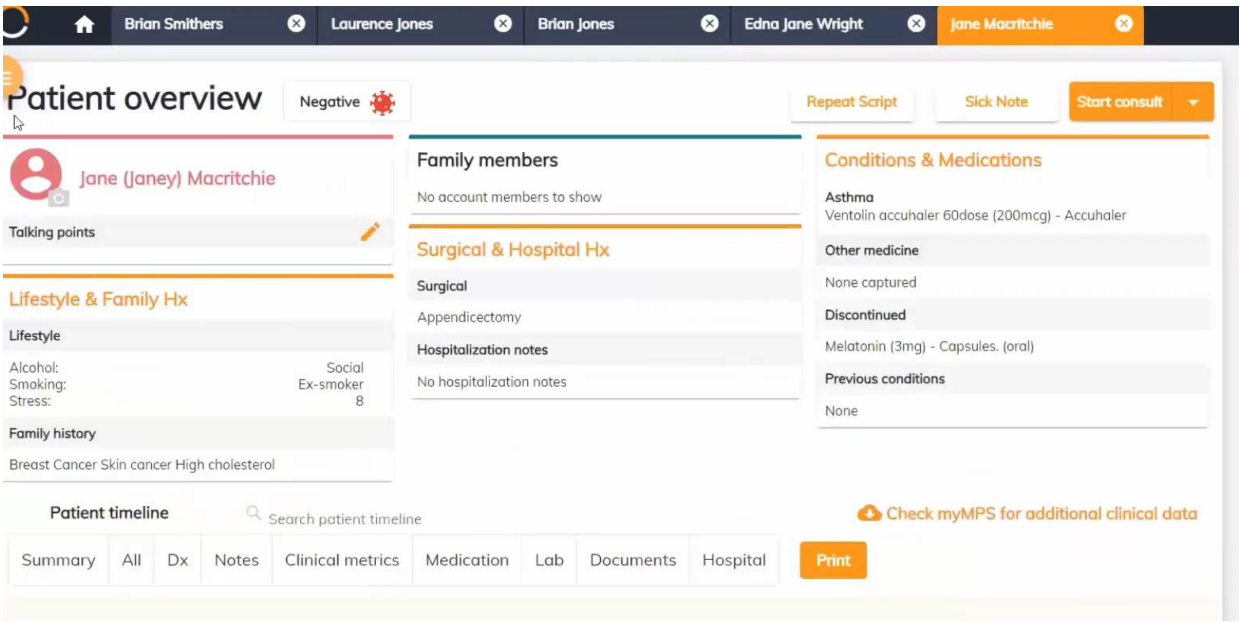
a. Describe the sequence of steps you followed during the virtual brain surgery simulation.
b. Observe and note how spatial awareness and hand-eye coordination were challenged in VR compared to your expectations of real surgery.
c. Reflect on how experience with VR surgical simulation could impact real-world surgical training and patient safety.

EXPERIMENT NO: 12

OBJECTIVE:
To design an electronic medical health record for a patient using Healthbridge simulation software.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Computer with internet connection, Web browser (Google Chrome/Mozilla Firefox), Healthbridge Clinical EMR software, Practice Management Application (PMA), Keyboard and Mouse for input, Monitor for display output, and Internet connection for cloud access

THEORY:
To design an electronic medical health record (EMR) for a patient using Healthbridge simulation software, it is important to understand the core concepts, functionality, and workflow integration of EMR systems in healthcare. An EMR system digitizes a patient's medical records, transforming traditional paper charts into an interactive and accessible digital platform tailored for clinical use. Healthbridge is a simulation-based software designed to replicate real clinical workflows by capturing patient information, clinical documentation, and administrative tasks efficiently. The primary goal of designing an EMR within Healthbridge is to model a comprehensive patient record that includes patient demographics, medical history, symptoms, diagnoses, treatment plans, lab results, and medication management. The system employs customizable templates and free-text options to document clinical encounters while maintaining standardization through clinical terminologies like ICD-10 and SNOMED CT. Healthbridge offers integrated features such as AI-assisted decision support, real-time data retrieval, telehealth consultation capabilities, and billing integration, all of which aim to enhance clinical efficiency and patient care quality. The design process involves setting up patient profiles, simulating consultations using workflow-optimized templates, documenting clinical notes via structured formats like SOAP, managing prescriptions electronically, and ensuring secure data access and compliance with healthcare regulations. Simulation with Healthbridge allows users to practice and refine EMR usage in a controlled environment without compromising real patient data, making it ideal for training and system testing. The software's cloud-based infrastructure supports remote accessibility, facilitating collaboration and continuous care coordination among healthcare providers.



EXPERIMENT PROCEDURE:

[OBSERVATION-01]

Pre-Consultation Setup and Administrative Integration

Step	Action	Observation
1	Open Healthbridge EMR system	Dashboard displays waiting room with patient list, status, and waiting times
2	Click on patient name	Patient file opens with automatic display of financial liability and account

		status
3	View Patient Executive Summary	Shows Medical Aid Plan, Allergies, Chronic Medications, and Lifestyle information

Step	Consultation Phase	System Function
1	Symptoms	COVID-19 template auto-populates common symptoms and questions
2	Examination	Template-driven physical examination with relevant fields for respiratory system
3	Diagnose & Prescribe	Favorites allow quick addition of diagnoses, procedures, and medications
4	Plan	Auto-generates digitally signed prescriptions, sick notes, and referral letters

1. Observe the four-step consultation process
2. Note template usage for COVID-19 case
3. Document favorites system for common entries
4. Record automated document generation process

RESULT:

Feature Tested	Result	Verification	Evidence
EMR-PMA Integration	Successful	Real-time admin data displayed in clinical view	Patient liability shown at file opening
Template Efficiency	Successful	Rapid data entry for common conditions	COVID-19 template auto-filled symptoms
Document Generation	Successful	Automatic creation of prescriptions and referrals	Digital scripts and referral letters created
Workflow Completion	Successful	End-to-end digital consultation achieved	Complete patient journey documented

Workflow Visualization:

Patient Arrival

↓

Waiting Room Management (PMA Integration)

↓

File Opening (Automatic Liability Display)

↓

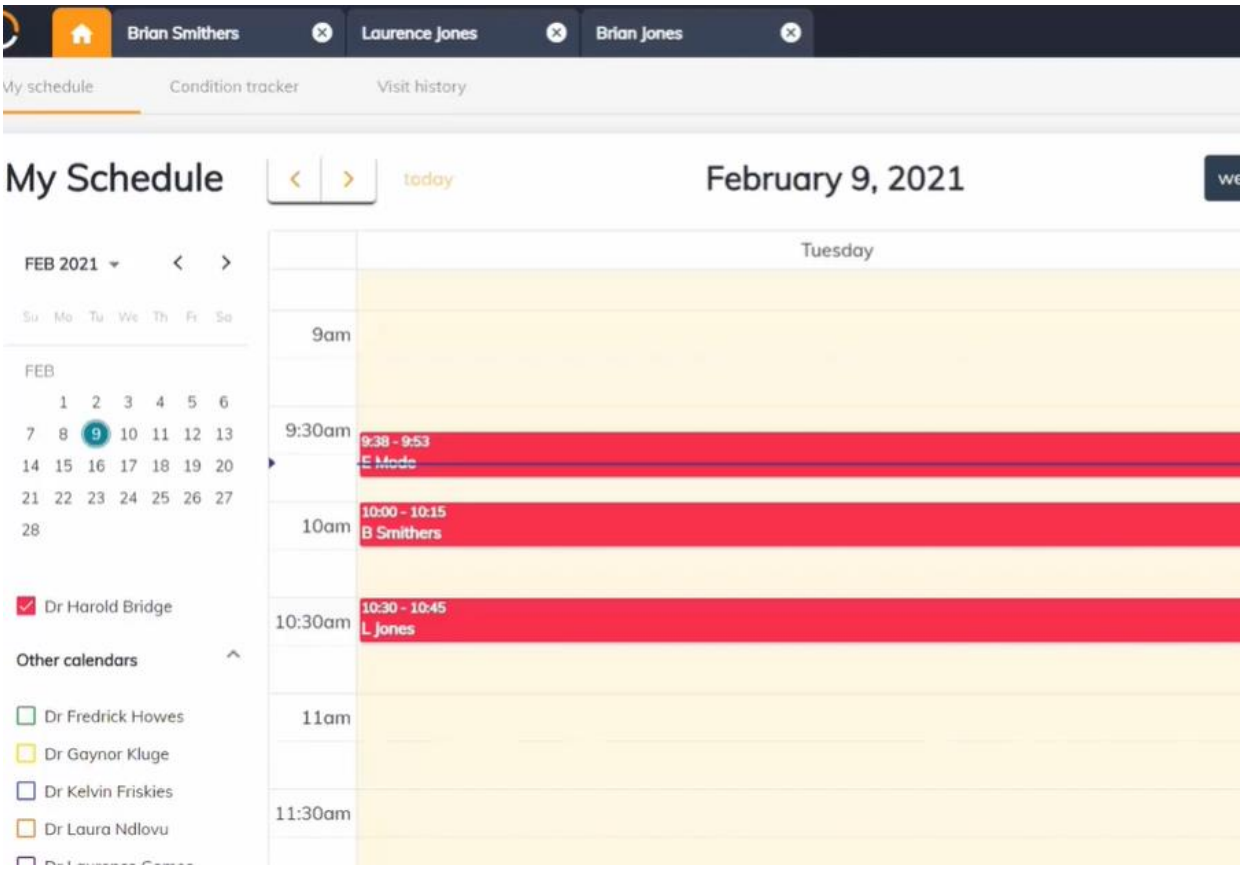
Consultation (4 Steps: Symptoms→Examination→Diagnose→Plan)

↓

Document Generation (Prescriptions/Referrals/Sick Notes)

↓

Visit Completion



PRE LAB QUESTIONS

1. What is an Electronic Medical Record (EMR) system, and how does it improve patient care compared to traditional paper records?
2. How does Healthbridge Clinical EMR integrate administrative tasks like billing and appointment scheduling with clinical workflows?
3. What are the main advantages of using template-driven data entry and the favorites system in an EMR?
4. Explain the importance of workflow integration in an EMR system and how digital consultation processes enhance clinical efficiency.
5. What security and privacy considerations must be addressed when designing and using an EMR system?

POST LAB QUESTIONS

1. How did the integration of administrative data (billing and appointment status) with clinical workflows improve the consultation process in Healthbridge EMR?
2. Describe how template-driven data entry and favorites affected the efficiency of clinical documentation during the simulated consultation.
3. What were the benefits of automated generation of prescriptions, sick notes, and referral letters in the Healthbridge system?
4. How does Healthbridge EMR support a stepwise transition from paper records to a fully digital workflow?

5. In what ways can real-time data access and workflow integration in Healthbridge EMR contribute to improved patient care and provider satisfaction?

EXERCISE

a. Describe the workflow integration observed in the Healthbridge Clinical EMR system between clinical consultation and administrative tasks.
b. Discuss the advantages and potential challenges of transitioning from paper-based records to digital EMR systems. How does Healthbridge facilitate this transition?
c. Reflect on your experience using the Healthbridge Clinical EMR simulation. What improvements would you suggest for the system to better serve healthcare providers and patients?

EXPERIMENT NO: 13

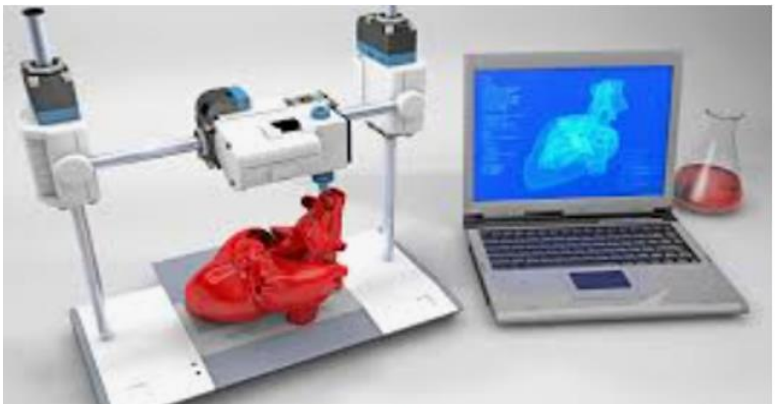
OBJECTIVE:
To perform the setting up of 3D Printing Lab

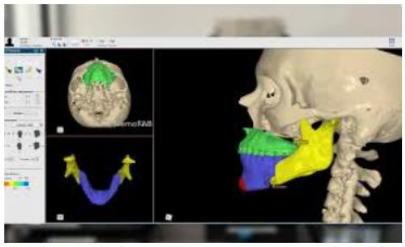
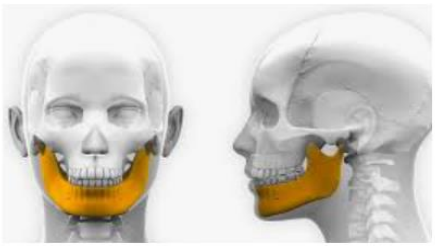
EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
3D printers (FDM, PolyJet, or other industrial-grade systems), Design and print preparation software (e.g., GrabCAD Print, Stratasys software), Computers/workstations in proximity to printers, Post-processing stations and cleaning areas, Proper ventilation and environmental controls (humidity, temperature), and Storage for materials and build plates, Support infrastructure: power, data connections, cooling, and vacuum systems.

THEORY:
3D printing in healthcare design is a cutting-edge technology that transforms digital models into physical objects through a precise, layer-by-layer material deposition process. It begins with acquiring detailed anatomical data from medical imaging techniques such as CT scans, MRIs, or ultrasound, which are then processed through segmentation to isolate specific structures of interest and generate accurate 3D digital models. These models serve as blueprints for the printing process, which involves selecting suitable biocompatible materials—such as polymers, metals, ceramics, or bio-inks—based on the specific medical application. The 3D printer constructs the object layer by layer, building complex shapes that would be difficult or impossible to create with traditional manufacturing methods. Post-processing steps like cleaning, curing, sterilizing, and finishing prepare the printed object for clinical use. This technology offers unprecedented customization, enabling the production of patient-specific medical devices, implants, surgical tools, anatomical models, and prosthetics that precisely match individual anatomy and pathology. This bespoke approach not only improves the fit and comfort of medical devices but also enhances surgical precision, reduces operation times, and minimizes complications. Moreover, 3D printing accelerates innovation and agility in product development by allowing rapid prototyping and design iteration, making it easier for medical professionals and engineers to collaborate and refine solutions based on immediate clinical feedback. Hospitals can centralize 3D printing services to streamline workflows and deliver on-demand manufacturing at the point of care, even extending advanced solutions to remote or underserved populations. With regulatory support and established clinical standards, 3D printing is revolutionizing healthcare by enabling personalized medicine, increasing efficiency, fostering continuous innovation, and ultimately improving patient outcomes in a range of medical fields.

EXPERIMENT PROCEDURE:
1. Select appropriate 3D printing technologies and materials based.
2. Evaluate and choose compatible software for design, slicing, and printer management.
3. Design the physical space considering printer placement, post-processing areas, collaborative workspaces, and storage.
4. Arrange for proper utilities including power supply, ventilation, air conditioning, humid-free zones, and data connectivity.
5. Plan for lab operations including staffing knowledgeable operators, scheduling print jobs, and managing materials.
6. Implement maintenance and support protocols to ensure printer longevity and workflow efficiency.
7. Evaluate sample prints from different systems to understand capabilities and reliability.
8. Integrate the lab into curricula, student clubs, and community partnership projects, ensuring hands-on learning opportunities.
9. Monitor lab usage, demand capacity, and manage growth opportunities for future expansion.

RESULTS




PRE LAB QUESTIONS

1. What are the primary factors to consider when planning an academic 3D printing lab, and why is defining the lab's purpose important?
2. How do the types of 3D printing technologies (such as FDM, PolyJet) influence the selection of equipment and materials for a lab?
3. Why is proper lab space design, including ventilation and environmental controls, critical for successful 3D printing operations?
4. What roles do software and workflow management play in the operation and efficiency of a 3D printing lab?
5. How can ongoing lab management, including staffing and maintenance, impact the sustainability and growth of a 3D printing lab?

POST LAB QUESTIONS

1. How did the purpose and defined goals of your 3D printing lab influence your choice of equipment and materials?
2. What challenges did you encounter in planning the lab space, and how did you address ventilation, power, and environmental control needs?
3. How does integrating software and workflow management contribute to the overall efficiency and user experience of the 3D printing lab?
4. What strategies did you propose for ongoing lab management, including staffing and maintenance, to ensure long-term sustainability?
5. How can the academic 3D printing lab facilitate community partnerships and student workforce development opportunities?

EXERCISE

Write one-page report describing your observations during the lab session, using your own words.

EXPERIMENT NO: 14

OBJECTIVE:
To understand the basic structure and functioning of a blockchain and implement a simple blockchain with multiple blocks using Python.

EQUIPMENT / SOFTWARE PACKAGE REQUIRED:
Computer with Python 3.x installed, Google Colab or Jupyter notebook for code execution and Internet connection to install required packages (if any).

THEORY:
Blockchain is a decentralized ledger which consists of blocks chained together via cryptographic hashes. Each block contains data, a timestamp, a nonce (for mining), and the hash of the previous block. Mining involves calculating a hash with a certain difficulty target (proof-of-work), ensuring tamper-resistance and security.

EXPERIMENT:

- Open the Google Colab
- Open the File
- Upload the Jupyter File from below link
- https://github.com/niaz1971/Digital_Transformation_Industry_4_5/blob/main/PRACTICAL_WORK_BOOK/CREATE_BLOCK_CHAIN.ipynb

```
import hashlib
import datetime
import networkx as nx
import matplotlib.pyplot as plt

class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        content = str(self.index) + str(self.timestamp) +
str(self.data) + str(self.previous_hash) + str(self.nonce)
        return hashlib.sha256(content.encode()).hexdigest()

    def mine_block(self, difficulty):
        prefix = '0' * difficulty
        while self.hash[:difficulty] != prefix:
            self.nonce += 1
            self.hash = self.calculate_hash()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 4

    def create_genesis_block(self):
        return Block(0, datetime.datetime.now(), "Genesis Block", "0")
```

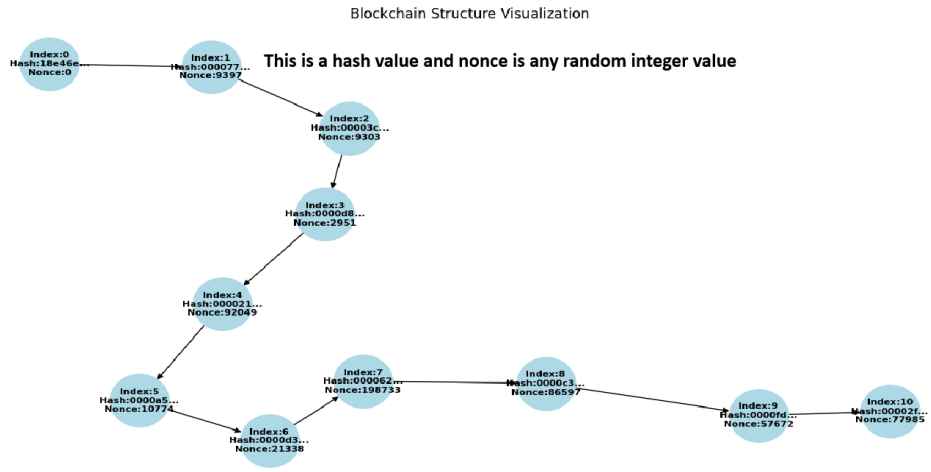
```
    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, new_block):
        new_block.previous_hash = self.get_latest_block().hash
        new_block.mine_block(self.difficulty)
        self.chain.append(new_block)

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            curr = self.chain[i]
            prev = self.chain[i-1]

            if curr.hash != curr.calculate_hash():
                return False
            if curr.previous_hash != prev.hash:
                return False
        return True

def plot_blockchain(blockchain):
    G = nx.DiGraph()

    for block in blockchain.chain:
        label =
f"Index:{block.index}\nHash:{block.hash[:6]}...\nNonce:{block.nonce}"
        G.add_node(block.index, label=label)

    for i in range(1, len(blockchain.chain)):
        G.add_edge(blockchain.chain[i-1].index,
blockchain.chain[i].index)

    pos = nx.spring_layout(G, seed=42)  # layout for neat
visualization
    labels = nx.get_node_attributes(G, 'label')
    plt.figure(figsize=(12,6))
    nx.draw(G, pos, with_labels=True, labels=labels, node_size=2500,
node_color='lightblue', font_size=8, font_weight='bold', arrows=True)
    plt.title("Blockchain Structure Visualization")
    plt.show()

# Main execution

my_blockchain = Blockchain()

for i in range(1, 11):
    print(f"Mining block {i}...")
    data = {"amount": i * 10}
    new_block = Block(i, datetime.datetime.now(), data, "")
    my_blockchain.add_block(new_block)

# Print blockchain details
for block in my_blockchain.chain:
```

```
    print(f"Index: {block.index}")
    print(f"Timestamp: {block.timestamp}")
    print(f"Data: {block.data}")
    print(f"Hash: {block.hash}")
    print(f"Previous Hash: {block.previous_hash}\n")

print("Is blockchain valid?", my_blockchain.is_chain_valid())

# Plot the blockchain structure
plot_blockchain(my_blockchain)
```

RESULT

```
Mining block 1...
Mining block 2...
Mining block 3...
Mining block 4...
Mining block 5...
Mining block 6...
Mining block 7...
Mining block 8...
Mining block 9...
Mining block 10...
Index: 0
Timestamp: 2025-10-30 17:59:46.296600
Data: Genesis Block
Hash: 18e46e25150c364f3f193ae04c0b9a35162d5521cd4cc4b71bd2055b08a91b3d
Previous Hash: 0

Index: 1
Timestamp: 2025-10-30 17:59:46.305104
Data: {'amount': 10}
Hash: 00007768f3adc0f93006a8d60e54badc5660ffdb82a53734a6e453b66ab0839b
Previous Hash: 18e46e25150c364f3f193ae04c0b9a35162d5521cd4cc4b71bd2055b08a91b3d

Index: 2
Timestamp: 2025-10-30 17:59:46.485511
Data: {'amount': 20}
Hash: 00003c19b5f2ec93479cf8709d6949cc56b1d3cb58d8b7590ed90ef0240b5c05
Previous Hash: 00007768f3adc0f93006a8d60e54badc5660ffdb82a53734a6e453b66ab0839b

Index: 3
Timestamp: 2025-10-30 17:59:46.595431
Data: {'amount': 30}
Hash: 0000d8379665798798fa28e252d7a88e55585f722e89c7de89dc6af1653dcee9
Previous Hash: 00003c19b5f2ec93479cf8709d6949cc56b1d3cb58d8b7590ed90ef0240b5c05

Index: 4
Timestamp: 2025-10-30 17:59:46.635753
Data: {'amount': 40}
Hash: 000021733073260c90dfb575c1ff04b57402b949c6f1ab67c65441ae1ac0c756
Previous Hash: 0000d8379665798798fa28e252d7a88e55585f722e89c7de89dc6af1653dcee9
```

```
Index: 5
Timestamp: 2025-10-30 17:59:47.316432
Data: {'amount': 50}
Hash: 0000a59c47d35db45e6182a36355ee28b95edbd46a4c6b9ee3e6d300257a6dbc
Previous Hash: 000021733073260c90dfb575c1ff04b57402b949c6f1ab67c65441ae1ac0c756

Index: 6
Timestamp: 2025-10-30 17:59:47.376522
Data: {'amount': 60}
Hash: 0000d3822060b73294b8acce620c6d334a96bd55c044dbe2fb2df43bba8b693d
Previous Hash: 0000a59c47d35db45e6182a36355ee28b95edbd46a4c6b9ee3e6d300257a6dbc

Index: 7
Timestamp: 2025-10-30 17:59:47.479743
Data: {'amount': 70}
Hash: 0000622fa210051990be9b45061f3f2fd377b754685ee39a19174f9a2a79ca2c
Previous Hash: 0000d3822060b73294b8acce620c6d334a96bd55c044dbe2fb2df43bba8b693d

Index: 8
Timestamp: 2025-10-30 17:59:48.859041
Data: {'amount': 80}
Hash: 0000c37bde816055095fb41f169582241e1b1ad64f0264e23c47a4c08c62f6fe
Previous Hash: 0000622fa210051990be9b45061f3f2fd377b754685ee39a19174f9a2a79ca2c

Index: 9
Timestamp: 2025-10-30 17:59:49.067672
Data: {'amount': 90}
Hash: 0000fd2d1610c9d1f19b3bf0130aece7c15ad80cc3a566b8d99f5a3b2f54dacd
Previous Hash: 0000c37bde816055095fb41f169582241e1b1ad64f0264e23c47a4c08c62f6fe

Index: 10
Timestamp: 2025-10-30 17:59:49.208740
Data: {'amount': 100}
Hash: 00002f3a78c1749a559b32f683f44065c443fb0a43896b241ae02a8874c9db4d
Previous Hash: 0000fd2d1610c9d1f19b3bf0130aece7c15ad80cc3a566b8d99f5a3b2f54dacd

Is blockchain valid? True
```



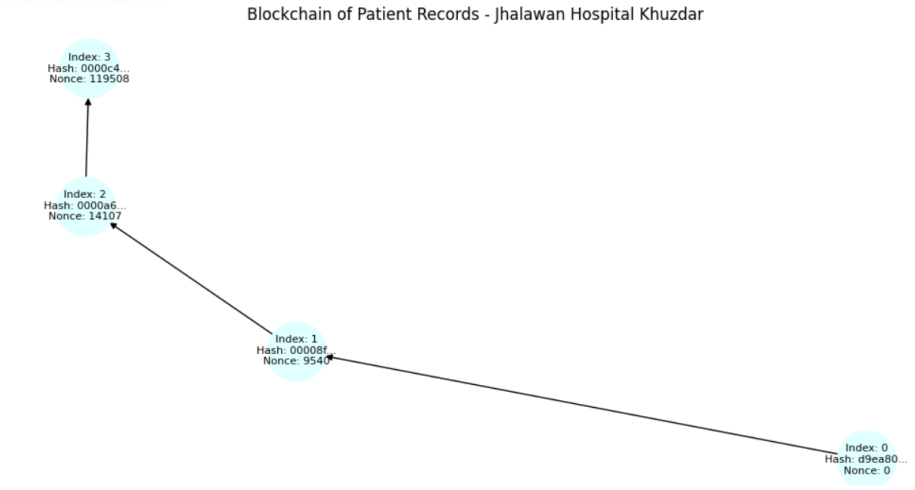
OBSERVATION

- Open the Google Colab
- Open the File
- Upload the Jupyter File from below link
  https://github.com/niaz1971/Digital_Transformation_Industry_4_5/blob/main/PRACTICAL_WORK_BOOK/CREATE_BLOCK_CHAIN_KHUZDAR_HOSPITAL.ipynb
- Run the Code

```
import hashlib
import datetime
import networkx as nx
import matplotlib.pyplot as plt

class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        content = str(self.index) + str(self.timestamp) + str(self.data) +
str(self.previous_hash) + str(self.nonce)
        return hashlib.sha256(content.encode()).hexdigest()

    def mine_block(self, difficulty):
        prefix = '0' * difficulty
        while self.hash[:difficulty] != prefix:
            self.nonce += 1
            self.hash = self.calculate_hash()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 4

    def create_genesis_block(self):
        return Block(0, datetime.datetime.now(), "Genesis Block", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, new_block):
        new_block.previous_hash = self.get_latest_block().hash
        new_block.mine_block(self.difficulty)
        self.chain.append(new_block)

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
```

```
            curr = self.chain[i]
            prev = self.chain[i-1]
            if curr.hash != curr.calculate_hash() or curr.previous_hash !=
prev.hash:
                return False
        return True

def plot_blockchain(blockchain):
    G = nx.DiGraph()
    for block in blockchain.chain:
        label = f"Index: {block.index}\nHash: {block.hash[:6]}...\nNonce:
{block.nonce}"
        G.add_node(block.index, label=label)
    for i in range(1, len(blockchain.chain)):
        G.add_edge(blockchain.chain[i-1].index, blockchain.chain[i].index)
    pos = nx.spring_layout(G, seed=42)
    labels = nx.get_node_attributes(G, 'label')
    plt.figure(figsize=(10,5))
    nx.draw(G, pos, with_labels=True, labels=labels, node_color='lightcyan',
node_size=2000, font_size=8, arrows=True)
    plt.title("Blockchain of Patient Records - Jhalawan Hospital Khuzdar")
    plt.show()

# Patient data for 3 blocks - Jhalawan Hospital Khuzdar
patient_data = [
    {"hospital": "Jhalawan Hospital Khuzdar", "patient_name": "Ali",
"diagnosis": "Flu"},
    {"hospital": "Jhalawan Hospital Khuzdar", "patient_name": "Muhammad",
"diagnosis": "Cold"},
    {"hospital": "Jhalawan Hospital Khuzdar", "patient_name": "Ahmed",
"diagnosis": "Diabetes"}
]

# Initialize blockchain and add blocks
blockchain = Blockchain()
for i, pdata in enumerate(patient_data, 1):
    print(f"Mining block {i} for patient {pdata['patient_name']}...")
    new_block = Block(i, datetime.datetime.now(), pdata, "")
    blockchain.add_block(new_block)

# Display blockchain details
for block in blockchain.chain:
    print(f"Index: {block.index} | Data: {block.data} | Hash: {block.hash}")

print("\nIs blockchain valid?", blockchain.is_chain_valid())

# Plot blockchain
plot_blockchain(blockchain)
```

```
Mining block 1 for patient Ali...
Mining block 2 for patient Muhammad...
Mining block 3 for patient Ahmed...
Index: 0 | Data: Genesis Block | Hash: d9ea80c5075d46cb50261173595a51db40ad5e807f7f6468cff7f68746c02531
Index: 1 | Data: {'hospital': 'Jhalawan Hospital Khuzdar', 'patient_name': 'Ali', 'diagnosis': 'Flu'} | Hash: 00008fff48fdac1de7d984bc901c9c26f4bad7b96e62fcf735d24df721e209d3
Index: 2 | Data: {'hospital': 'Jhalawan Hospital Khuzdar', 'patient_name': 'Muhammad', 'diagnosis': 'Cold'} | Hash: 0000a666969aae4c753a4eb761fce9616e4ac3affe1f1c1fdd4f299822ddf893
Index: 3 | Data: {'hospital': 'Jhalawan Hospital Khuzdar', 'patient_name': 'Ahmed', 'diagnosis': 'Diabetes'} | Hash: 0000c48070ff922b19f40b186b816d9362a0f7e651f25ace0d2677a1a5a134f8

Is blockchain valid? True
```



EXERCISE:

Write a Python and sets up a blockchain for five patients of any hospital near to your city, write five patients names, create the mining blocks, indexes, also verify is it blockchain valid? And finally draw blockchain structure.